

控制单元操作

学习目标

学习本章后，你将可以：

- 解释微操作的概念，并根据微操作定义主要的指令周期阶段。
- 讨论如何组织微操作来扩展处理器。
- 理解硬布线控制单元组织结构。

第 12 章指出，机器指令集对定义处理器有很大帮助。如果知道机器指令集，包括对每个操作码效果和对寻址模式的理解，如果了解用户可见寄存器组，那么我们就知道处理器必须执行的功能。这不是全部的情况。我们必须知道通常通过总线的外部接口，以及如何处理中断。有了这样的推理，下面列出了指定处理器功能所需的内容：

- 1) 操作（操作码）。
- 2) 寻址模式。
- 3) 寄存器。
- 4) I/O 模块接口。
- 5) 内存模块接口。
- 6) 中断。

这个列表虽然比较笼统，但是很完整。第一条到第三条由指令集定义。第四条和第五条通常是由指定系统总线来定义。第六条部分由系统总线定义，部分由处理器提供给操作系统的支持类型来定义。

这 6 条可以称为处理器的功能需求。它们决定了处理器必须做什么。这就是第二部分和第四部分所讨论的内容。现在，我们将讨论这些功能是如何执行的，或者更具体地说，如何控制处理器的各个元素来提供这些功能。因此，我们转而讨论控制单元，它控制着处理器的操作。

20.1 微操作

我们已经看到，在执行程序的时候，计算机的操作是由一系列的指令周期组成的，每个周期一条机器指令。当然，我们必须记住，由于存在分支指令，这个指令周期序列不一定要与组成程序的书面指令序列顺序一致。这里，我们指的是指令的执行时间序列。

进一步，我们还看到，每个指令周期都是由一些更小的单元组成的。我们发现的一个方便的细分是取指、间接、执行和中断，只有获取和执行周期总是发生。

但是，为了设计一个控制单元，我们需要进一步细化描述。在第 14 章讨论流水线时，我们开始看到进一步细化是可能的。实际上，我们将看到每个更小的周期都涉及一系列步骤，每个步骤都涉及处理器寄存器。我们把这些步骤称为微操作（micro-operation）。前缀 micro 指的是每一步都很简单，完成的很少。图 20-1 描述了我们所讨论的各种概念之间的

关系。总而言之，程序的执行是由指令的顺序执行构成的。每条指令都在一个指令周期中执行，该指令周期又是由更短的子周期（如：取指、间接、执行、中断）组成。每个子周期的执行都包含一个或多个更短的操作，即微操作。

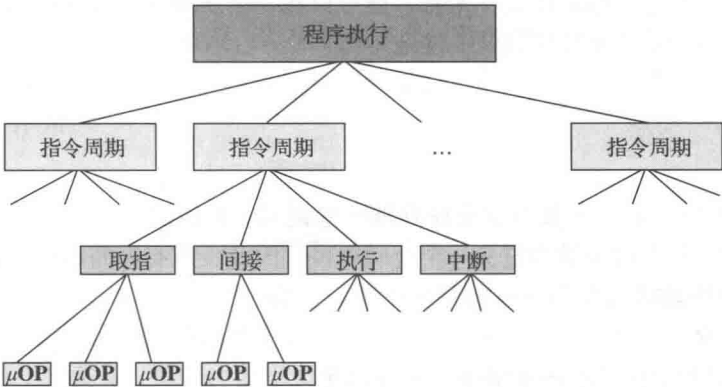


图 20-1 程序执行的组成要素

微操作是处理器的功能性或原子性操作。在本节中，我们将研究微操作，以了解如何将任意指令周期的事件描述为一系列此类微操作。我们将使用一个简单的例子。然后，在本章的其余部分，我们将展示微操作概念是如何指导设计控制单元的。

20.1.1 取指周期

我们首先来看取指周期，它发生在每个指令周期的开始，使得一条指令从内存中取出。出于讨论的目的，我们假设组织结构如图 14-6 所示。这涉及 4 个寄存器：

- 内存地址寄存器（MAR）与系统总线的地址线相连。它指定读或写操作的内存地址。
- 内存缓冲寄存器（MBR）与系统总线的数据线相连。它包含了要存储到内存中的数值，或从内存中读出的最后一个数值。
- 程序计数器（PC）保存下一条要取的指令的地址。
- 指令寄存器（IR）保存最后获取的指令。

让我们从对处理器寄存器的影响的角度来看看取指周期的事件序列。图 20-2 给出了一个例子。在取指周期开始的时候，下一条将要执行指令的地址在程序计数器中（PC）；在本例中，地址为 1100100。第一步是把该地址送入内存地址寄存器（MAR），因为它是唯一连接系统总线中地址线的寄存器。第二步是取入指令。所需地址（在 MAR 中）被放到地址总线上，控制单元向控制总线发出 READ 命令，结果出现在数据总线上并被复制到内存缓冲寄存器（MBR）。我们还需要按指令长度增加 PC，以便为下一条指令做准备。由于这两个操作（从内存中读取字、增加

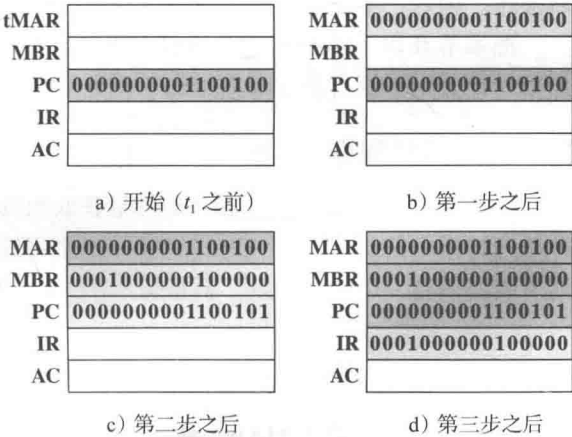


图 20-2 事件序列，取指周期

PC) 互不干扰, 它们可以同时进行, 以节省时间。第三步是把 MBR 中的内容送入指令寄存器 (IR)。这会释放 MBR, 以便在可能的间接周期中使用。

因此, 简单的取指周期实际上包含了 3 个步骤和 4 个微操作。每个微操作都涉及数据移入和移出寄存器。只要这些动作互不干扰, 就可以在一个步骤中进行其中的几个动作, 以节省时间。我们可以把这个事件序列象征性地写为

```
t1: MAR ← (PC)
t2: MBR ← Memory
    PC ← (PC) + I
t3: IR ← (MBR)
```

其中, I 是指令长度。我们需要对这个序列做一些说明。假设时钟可用来计时, 它定期发出间隔的时钟脉冲。每个时钟脉冲定义一个时间单位。因此, 所有时间单位都具有相同的持续时间。每个微操作都可以在一个时间单位中执行。符号 (t_1, t_2, t_3) 表示连续的时间单位。换句话说, 我们有

- 第一个时间单位: 把 PC 的内容送入 MAR。
- 第二个时间单位: 把由 MAR 指定的内存位置中的内容送入 MBR。PC 内容加 I 。
- 第三个时间单位: 把 MBR 的内容送入 IR。

注意, 第二个和第三个微操作都是在第二个时间单位中发生的。第三个微操作可以与第四个微操作组合在一起, 且不会影响取指操作:

```
t1: MAR ← (PC)
t2: MBR ← Memory
t3: PC ← (PC) + I
    IR ← (MBR)
```

微操作分组必须遵循两个简单的规则:

1) 必须遵循正确的事件顺序。所以, $(MAR \leftarrow (PC))$ 必须在 $(MBR \leftarrow Memory)$ 之前, 因为内存读操作要使用 MAR 中的地址。

2) 必须避免冲突。不能在一个时间单位中尝试读写同一个寄存器, 因为其结果是不可预测的。例如, 微操作 $(MBR \leftarrow Memory)$ 和 $(IR \leftarrow MBR)$ 就不能发生在同一个时间单位中。

最后一点值得注意的是, 其中的一个微操作涉及加法。为了避免电路的重复, 这个加法可以由 ALU 来执行。根据 ALU 的功能和处理器的组织结构, ALU 的使用可能涉及额外的微操作。稍后, 本章将对这一点进行讨论。

把本节及以下小节描述的事件与图 3-5 进行比较是很有用的。虽然该图忽略了微操作, 但本讨论显示了执行指令周期中的子周期所需要的微操作。

20.1.2 间接周期

一旦取到了一条指令, 下一步就是获取源操作数。继续我们的简单例子, 让我们假设一种单地址指令格式, 该格式允许直接和间接寻址。如果指令指定了间接地址, 那么在执行周期前就必须有一个间接周期。数据流与图 14-7 所示的数据流有所不同, 包含了以下微操作:

```
t1: MAR ← (IR(Address))
t2: MBR ← Memory
t3: IR(Address) ← (MBR(Address))
```

指令的地址字段被送入 MAR。然后, 用它来获取操作数地址。最后, 从 MBR 更新 IR 的地址字段, 使其现在包含一个直接地址, 而不是一个间接地址。

现在, IR 的状态与没有使用间接地址的状态是一样的, 而且已经为执行周期做好了准备。我们暂时跳过这个周期, 考虑中断周期。

20.1.3 中断周期

执行周期完成后, 要进行检测, 以确定是否发生了任何允许的中断。如果是, 则发生中断周期。这个周期的性质在不同机器间有着很大的差异。如图 14-8 所示, 我们提供了一个非常简单的事件序列:

```
t1: MBR ← (PC)
t2: MAR ← Save_Address
    PC ← Routine_Address
t3: Memory ← (MBR)
```

在第一步中, PC 的内容被送入 MBR, 这样就可以保存它们, 以便从中断返回。然后, 把 PC 内容的保存地址加载到 MAR 中, 把中断处理例程的起始地址加载到 PC 中。这两个动作可能都是单个微操作。但是, 由于大多数处理器提供多种类型和级别的中断, 所以可能需要一个或多个额外的微操作来获得 Save_Address 和 the Routine_Address, 然后, 再把它们分别送给 MAR 和 PC。任何情况下, 一旦完成这一步, 最后一步就是把 MBR 存入内存, MBR 中包含的是 PC 的旧值。处理器现在准备开始下一个指令周期了。

20.1.4 执行周期

取指、间接和中断周期简单且可预测。每个周期都涉及小的、固定的微操作序列, 在每种情况下, 每次都会重复相同的微操作。

但执行周期却不是这样。由于操作码的多样性, 可能会发生许多微操作的不同序列。控制单元检查操作码, 并根据操作码的值生成微操作序列。这被称为指令译码。

让我们考虑几个假设的例子。

首先考虑一条加法指令:

```
ADD R1, X
```

它把位置 X 中的内容与寄存器 R1 的内容相加。可能会发生如下微操作序列:

```
t1: MAR ← (IR(address))
t2: MBR ← Memory
t3: R1 ← (R1) + (MBR)
```

我们从包含 ADD 指令的 IR 开始。第一步中, IR 的地址部分被加载到 MAR。然后读取被引用内存位置。最后, 用 ALU 实现 R1 的内容加上 MBR 的内容。同样, 这也是一个简化的例子。可能需要额外的微操作从 IR 提取寄存器引用, 并且可能在某些中间寄存器中暂存 ALU 的输入或输出。

让我们来看看两个更复杂的例子。增加且为零就跳过是常见指令:

```
ISZ X
```

位置 X 的内容加 1。如果其结果为 0, 则跳过下一条指令。可能的微操作序列为

```
t1: MAR ← (IR(address))
t2: MBR ← Memory
t3: MBR ← (MBR) + 1
t4: Memory ← (MBR)
    If ((MBR) = 0) then (PC ← (PC) + 1)
```


这里引入的新特性是条件动作。如果 $(MBR) = 0$ ，则增加 PC。这个测试和动作可以用一个微操作实现。还要注意的，可以在把 MBR 中更新值存回内存的同一个时间单位中执行这个微操作。

最后，考虑一个子例程调用指令。举个例子，考虑一条分支并保存地址的指令：

BSA X

BSA 指令后面的那条指令的地址保存到位置 X，并继续在位置 $X+I$ 执行。保存的地址稍后用于返回。这是一种支持子例程调用的简单技术。以下微操作就足够了：

```
t1: MAR ← (IR(address))  
      MBR ← (PC)  
t2: PC ← (IR(address))  
      Memory ← (MBR)  
t3: PC ← (PC) + I
```

指令开始时 PC 中的地址是下一条顺序指令的地址。它将保存在 IR 中指定的地址。后一个地址也会递增，以提供下一个指令周期的指令地址。

20.1.5 指令周期

我们已经看到，指令周期的每一个阶段都能分解成基本微操作序列。在我们的例子中，取指、间接和中断周期各有一个序列，而执行周期则是每个操作码有一个微操作序列。

为了完成整个过程，我们需要把这些微操作序列连接起来，如图 20-3 所示。我们假设有一个新的 2 位寄存器，称为指令周期码 (ICC)。ICC 根据处理器处于周期的哪个部分来指定处理器状态：

- 00: 取指
- 01: 间接
- 10: 执行
- 11: 中断

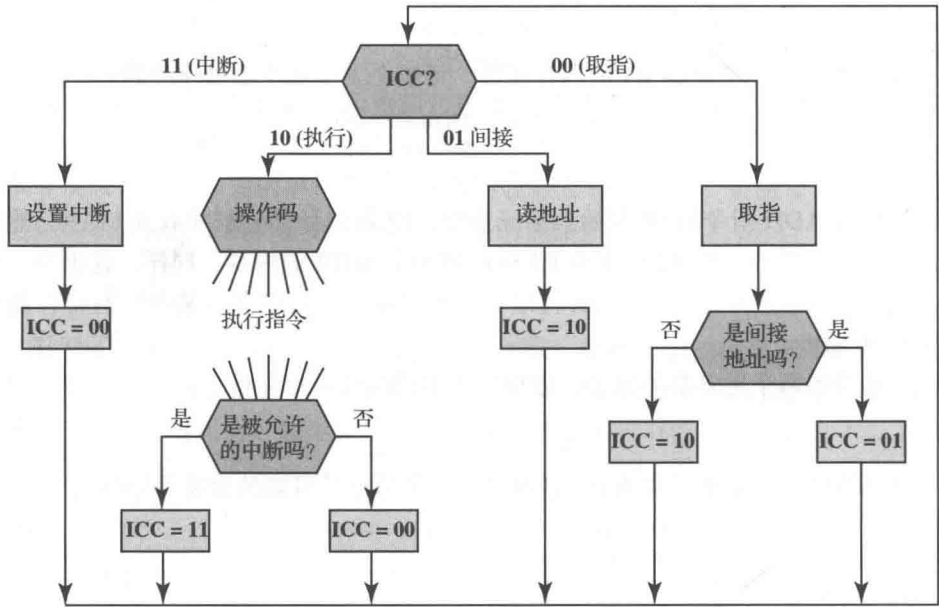


图 20-3 指令周期流程图

对这 4 个周期来说,在每个周期结束时,都会适当设置 ICC。间接周期的后面总是跟着执行周期。中断周期的后面总是跟着取指周期(参见图 14-4)。对于取指和执行周期,下一个周期取决于系统状态。

因此,图 20-3 的流程图定义了完整的微操作序列,它只依赖于指令序列和中断模式。当然,这是一个简化的例子。实际处理器的流程图要复杂得多。无论如何,我们已经达到了讨论点,即处理器的操作被定义为一系列微操作的性能。现在,我们可以考虑控制单元是如何使得这个序列发生的。

20.2 处理器的控制

20.2.1 功能需求

根据上一节分析的结果,我们已经把处理器的行为或功能分解为基本操作,称为微操作。通过把处理器的操作降到最基本的级别,我们可以准确地定义控制单元必然导致的操作。因此,我们可以为控制单元定义功能需求:控制单元必须执行的那些功能。这些功能需求的定义是设计和实现控制单元的基础。

有了这些信息后,以下三个步骤就可以确定控制单元的特性:

- 1) 定义处理器的基本要素。
- 2) 描述处理器执行的微操作。
- 3) 确定控制单元必须执行的功能,以执行微操作。

我们已经执行了步骤 1 和步骤 2。让我们总结一下结果。首先,处理器的基本功能要素如下:

- ALU
- 寄存器
- 内部数据通路
- 外部数据通路
- 控制单元

某些想法会让你相信这是一个完整的列表。ALU 是计算机功能的本质。寄存器用于存储处理器内部的数据。有些寄存器包含了管理指令排序所需的状态信息(如:程序状态字)。其他部分则包含进出 ALU、内存和 I/O 模块的数据。内部数据通路用于在寄存器之间和寄存器与 ALU 之间传输数据。外部数据通路把寄存器连接到内存和 I/O 模块,一般是通过系统总线。控制单元使操作在处理器内发生。

程序的执行由涉及这些处理器要素的操作组成。正如我们所看到的,这些操作由一系列微操作构成。回顾 20.1 节,读者应该看到,所有的微操作都属于以下类别中的一类:

- 把数据从一个寄存器传输到另一个寄存器。
- 把数据从寄存器传输到外部接口(如:系统总线)。
- 把数据从外部接口传输到寄存器。
- 执行算术或逻辑操作,使用寄存器输入/输出。

执行一个指令周期所需的所有微操作(包括执行指令集中每条指令所需的所有微操作)都属于这些类别中的一类。

现在,我们可以更明确地了解控制单元的工作方式。控制单元执行两个基本任务:

- **排序** 根据正在执行的程序，控制单元使处理器按正确的顺序执行一系列微操作。
- **执行** 控制单元使得每个微操作得以执行。

前面是控制单元的功能描述。控制单元如何工作的关键是控制信号的使用。

20.2.2 控制信号

我们已经定义了构成处理器的要素（ALU、寄存器、数据通路）和执行的微操作。为了控制单元能执行其功能，它必须有输入来使其决定系统状态，有输出来使其控制系统行为。这些都是控制单元的外部参数。在内部，控制单元必须具有实现其排序和执行功能所需的逻辑。稍后我们将在 20.3 节和第 21 章讨论控制单元内部操作。本节其余部分涉及控制单元和处理器其他要素之间的交互。

图 20-4 是控制单元的一般模型，显示了它所有的输入和输出。输入有

- **时钟**：这是控制单元“计时”的方式。控制单元使得每个时钟脉冲执行一个微操作（或一组同步微操作）。有时这也被称为处理器周期时间，或时钟周期时间。
- **指令寄存器**：当前指令的操作码和寻址模式，用于决定在执行周期中应执行哪些微操作。
- **标志**：控制单元需要这些值来决定处理器状态和之前 ALU 操作的结果。例如，对于增量且为 0 跳过（ISZ）指令，如果零标志置 1，则控制单元将增加 PC。

输出有：

- **从控制总线来的控制信号**：系统总线中的控制总线部分，向控制单元提供信号。
- **处理器内部的控制信号**：这有两种类型——使得数据从一个寄存器移动到另一个寄存器的类型和激活特定 ALU 功能的类型。
- **到控制总线去的控制信号**：这也有两种类型——发给内存的控制信号和发给 I/O 模块的控制信号。

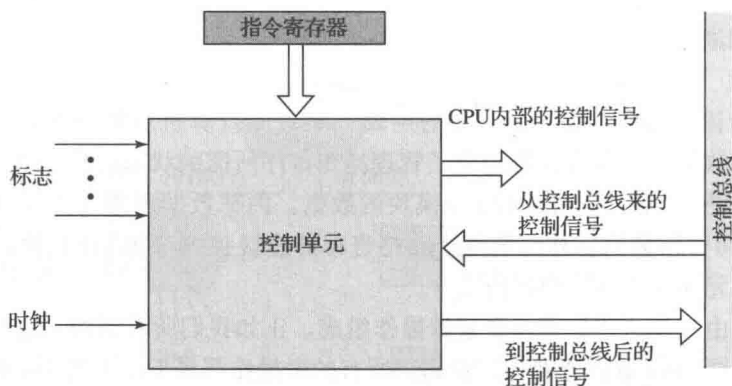


图 20-4 控制单元

使用了三种类型的控制信号：激活 ALU 功能的信号；激活数据通路的信号；外部系统总线或其他内部接口上的信号。所有这些信号最终都会直接作为二进制输入进入各个逻辑门。

让我们再次考虑取指周期，看看控制单元如何维持控制。控制单元跟踪它在指令周期中的位置。在给定的点上，它知道接下来要执行取指周期。第一步是把 PC 的内容传递给 MAR。控制单元通过激活控制信号打开 PC 位和 MAR 位之间门来实现这一步。下一步是把一个字从内存读入 MBR，并增加 PC。控制单元通过同时发送以下控制信号来完成这一步：

- 打开逻辑门的控制信号，允许 MAR 的内容送到地址总线上。
- 控制总线上的内存读控制信号。
- 打开逻辑门的控制信号，允许数据总线的内容存储到 MBR 中。
- 到逻辑的控制信号，把 PC 内容加 1，并将结果存回 PC。

之后，控制单元发送一个控制信号，打开 MBR 和 IR 之间的门。

这样就完成了取指周期，但有一件事情除外：控制单元必须决定下一步是执行一个间接周期还是一个执行周期。为此，它要检查 IR，看看是否有间接内存引用。

间接周期与中断周期的工作方式类似。对于执行周期，控制单元首先检查操作码，在此基础上，决定执行周期要执行的微操作序列。

20.2.3 控制信号示例

为了说明控制单元的功能，让我们看一个简单的例子。图 20-5 说明了这个例子。这是一个简单的处理器，带有一个累加器 (AC)。组件之间的数据通路也标识出来了。没有显示控制单元发出信号的控制通路，但控制信号的终端标记为 C_i ，并用圆圈表示。控制单元接收来自时钟、IR 和标志的输入。在每个时钟周期，控制单元读取其所有的输入并发出一组控制信号。控制信号发送到三个不同的目的地：

- **数据通路** 控制单元控制内部数据流。例如，在取指时，内存缓冲寄存器的内容被传送到 IR。对要控制的每个通路，都有一个开关（图中用圆圈表示）。来自控制单元的控制信号暂时打开门以便让数据通过。
- **ALU** 控制单元用一组控制信号来控制 ALU 的操作。这些信号激活 ALU 中的各种逻辑电路和门。
- **系统总线** 控制单元把控制信号发送到系统总线的控制线上（如：内存读）。

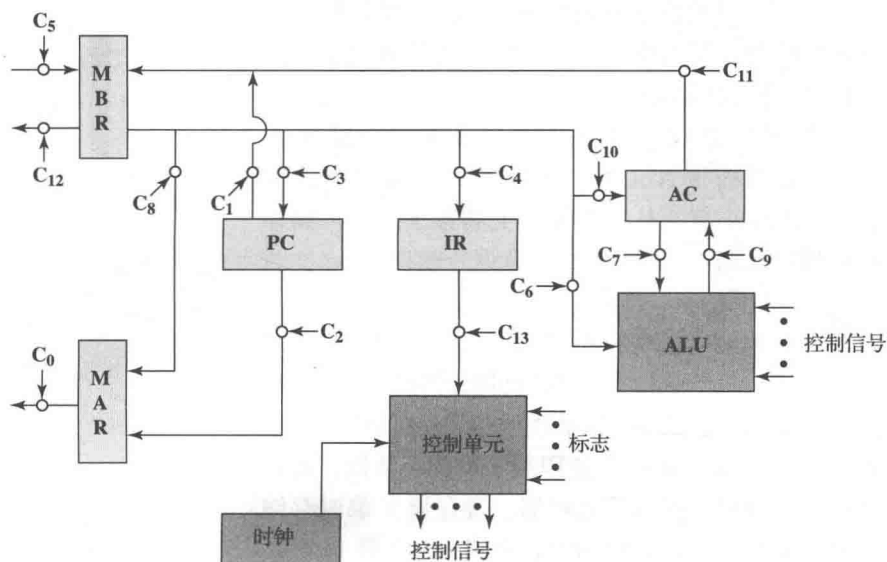


图 20-5 数据通路和控制信号

控制单元必须知道它在指令周期中的位置。利用这些信息，并通过读取其所有输入，控制单元发出一系列控制信号，使得微操作得以发生。它使用时钟脉冲对事件序列进行计时，使事

件之间的时间在信号级别上稳定下来。表 20-1 列出了前面描述的一些微操作序列所需的控制信号。为简单起见，没有给出增加 PC 以及向 PC 与 MAR 加载固定地址的数据和控制通路。

表 20-1 微操作和控制信号

	微操作	激活控制信号
取指	$t_1: MAR \leftarrow (PC)$	C_2
	$t_2: MBR \leftarrow Memory$ $PC \leftarrow (PC) + 1$	C_5, C_R
	$t_3: IR \leftarrow (MBR)$	C_4
间接	$t_1: MAR \leftarrow (IR(Address))$	C_8
	$t_2: MBR \leftarrow Memory$	C_5, C_R
	$t_3: IR(Address) \leftarrow (MBR(Address))$	C_4
中断	$t_1: MBR \leftarrow (PC)$	C_1
	$t_2: MAR \leftarrow Save-address\ s$ $PC \leftarrow Routine-address\ s$	
	$t_3: Memory \leftarrow (MBR)$	C_{12}, C_W

注： C_R = 到系统总线的读控制信号。
 C_W = 到系统总线的写控制信号。

值得考虑的是控制单元的最小特性。控制单元是运行整个计算机的引擎。它只根据知道要执行的指令以及算术和逻辑操作结果的性质（如：正数、溢出等）来执行。它永远不会看到正在处理的数据或实际产生的结果。它通过对处理器内各点的一些控制信号以及对系统总线的一些控制信号来控制一切。

20.2.4 处理器内部组织结构

图 20-5 显示了各种数据通路的使用。这种类型组织结构的复杂性应该很清楚。更为典型的是，将使用某种内部总线布局，如图 14-2 所建议的那样。

使用内部处理器总线，可以按图 20-6 来重新安排图 20-5。单一内部总线连接 ALU 和所有的处理器寄存器。提供门和控制信号用于把数据从寄存器移上和移下总线。额外的控制信号控制对系统（外部）总线的数据传输以及 ALU 的操作。

标记为 Y 和 Z 的两个新寄存器被添加到这个结构中。这些是 ALU 正常运行所需要的。当要执行的操作涉及两个操作数时，一个可以从内部总线获得，而另一个则必须从另一个源获得。AC 可以用于此目的，但这限制了系统灵活性，而且不适用于具有多个通用寄存器的处理器。寄存器 Y 临时存储其他输入。ALU 是没有内部存储的组合电路（参见第 11 章）。因此，当控制信号激活 ALU 功能时，ALU 的输入就被转换成输出。所以，ALU 的输出不能直接连接到总线，因为这个输出可能会反馈到输入。寄存器 Z 提供临时的输出存储。有了这种安排，把一个内存值加到 AC 的操作将会包含如下步骤：

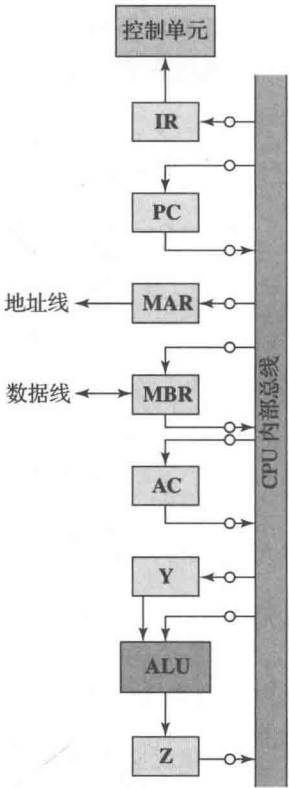


图 20-6 具有内部总线的 CPU

$$\begin{aligned} t_1: \text{MAR} &\leftarrow (\text{IR}(\text{address})) \\ t_2: \text{MBR} &\leftarrow \text{Memory} \\ t_3: Y &\leftarrow (\text{MBR}) \\ t_4: Z &\leftarrow (\text{AC}) + (Y) \\ t_5: \text{AC} &\leftarrow (Z) \end{aligned}$$

其他组织结构也是可以的，但是通常会使用某种内部总线或一组内部总线。使用通用数据通路简化了互连布局和处理器的控制。使用内部总线的另一个实际原因是节省了空间。

20.2.5 Intel 8085

为了说明本章到目前为止介绍的一些概念，让我们考虑一下 Intel 8085。它的组织结构如图 20-7 所示。下面几个关键组件可能不是那么一目了然：

- **递增 / 递减地址锁存器** 使栈指针或程序计数器内容加 1 或减 1 的逻辑。通过避免为此目的使用 ALU 来节省时间。
- **中断控制** 此模块处理多级中断信号。
- **串行 I/O 控制** 此模块与每次通信 1 位的设备相连。

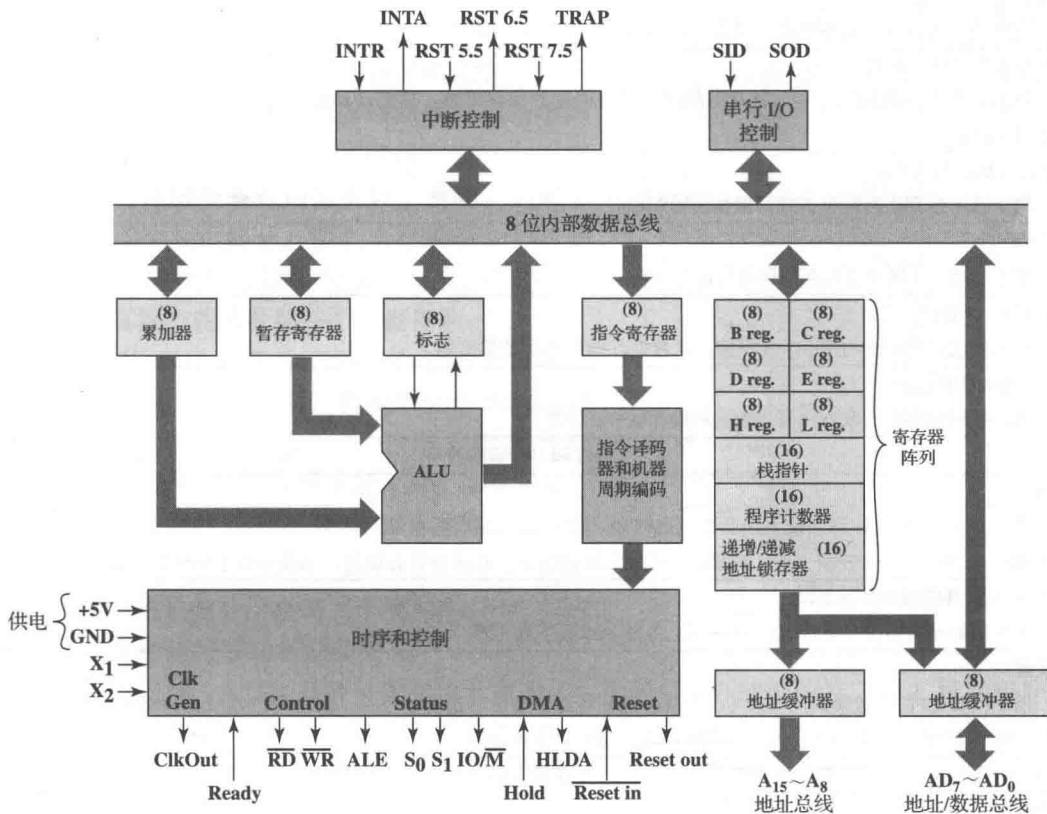


图 20-7 Intel 8085 CPU 方框图

表 20-2 描述了进出 8085 的外部信号。它们被链接到外部系统总线。这些信号是 8085 处理器和系统其余部分之间的接口 (图 20-8)。

该控制单元被识别为有两个组件，分别标记为：1) 指令译码器和机器周期编码；2) 时序和控制。下一节将讨论第一个组件。控制单元的实质是时序和控制模块。该模块包含一个

时钟，并接受当前指令和一些外部控制信号作为输入。它的输出包括发送到处理器其他组件的控制信号和发送到外部系统总线的控制信号。

表 20-2 Intel 8085 的外部信号

地址和数据信号
高地址 (A ₁₅ -A ₈) 16 位地址的高 8 位
地址 / 数据 (AD ₇ -AD ₀) 16 位地址的低 8 位或 8 位数据。为了节省引脚而复用
串行输入数据 (SID) 1 位输入，以适应串行发送的设备 (一次一位)
串行输出数据 (SOD) 1 位输出，以适应串行接收的设备
时序和控制信号
CLK (OUT) 系统时钟。CLK 信号进入外围芯片并同步它们的时序
X ₁ , X ₂ 这些信号来自外部晶振或其他设备，以驱动内部时钟发生器
地址锁存使能 (ALE) 发生在机器周期的第一个时钟状态期间，使外围芯片保存地址。这能让地址模块 (如：内存、I/O) 识别出它正在被寻址
状态 (S ₀ , S ₁) 用于指示是否正在发生读或写操作的控制信号
IO/M 用于启动 I/O 模块或内存模块进行读写操作
读控制 (RD) 指示要读取的选定内存或 I/O 模块，且数据总线可用于数据传输
写控制 (WR) 指示数据总线上的数据将写入选定的内存或 I/O 位置
内存和 I/O 启动符号
保持 请求 CPU 放弃对外部系统总线的控制和使用。CPU 将完成当前在 IR 中的指令的执行，然后进入保持状态，在此期间，CPU 不会向控制、地址或数据总线插入任何信号。在保持状态期间，总线可用于 DMA 操作
保持应答 (HOLDA) 该控制单元输出信号应答 HOLD 信号，并指示总线当前可用
就绪 用于同步 CPU 和较慢的内存或 I/O 设备。当被寻址设备发出就绪信号时，CPU 可以继续执行输入 (DBIN) 或输出 (WR) 操作。否则，CPU 将进入等待状态，直到设备准备好
中断相关的信号
陷阱 重启中断 (RST 7.5, 6.5, 5.5)
中断请求 (INTR) 外部设备使用这 5 根线来中断 CPU。如果 CPU 处于保持状态或中断被禁止，则 CPU 将不接受请求。只有一条指令完成时才会执行中断。中断按优先级降序排列
中断应答 应答一个中断

(续)

CPU 初始化

复位输入

使 PC 的内容被设置为 0。CPU 从位置 0 恢复执行

复位输出

确认 CPU 已经复位。该信号可用于复位系统的其余部分

电压和接地

 V_{CC}

+5V 电源

 V_{SS}

电气接地

处理器操作的时序由时钟同步，并由控制单元用控制信号来控制。每个指令周期被分为 1 ~ 5 个机器周期；每个机器周期又被分为 3 ~ 5 种状态。每种状态持续一个时钟周期。在一个状态期间，处理器执行由控制信号决定的一个或一组同步微操作。

对于给定指令，其机器周期数是固定的，但不同指令的机器周期数不同。机器周期被定义为等同于总线访问。所以，一条指令的机器周期数取决于处理器必须与外部设备通信的次数。例如，如果一条指令包含两个 8 位部分，那么取出该指令就需要两个机器周期。如果该指令包含 1 字节内存或 I/O 操作，那么其执行就需要第三个机器周期。

图 20-9 是 8085 时序的例子，它显示了外部控制信号值。当然，与此同时，控制单元生成内部控制信号来控制内部数据传输。图中给出了 OUT 指令的指令周期。需要三个机器周期 (M_1 , M_2 , M_3)。在第一个机器周期中，取指 OUT 指令。第二个机器周期取出该指令的第二部分，其中包含了进行输出的选定 I/O 设备的编号。在第三个周期中，AC 的内容通过数据总线被写到选定设备中。

地址锁存使能 (ALE) 脉冲从控制单元发出每个机器周期开始的信号。ALE 脉冲向外部电路发出警报。在机器周期 M_1 的时序状态 T_1 中，控制单元设置 IO/M 信号来表示这是一个内存操作。同时，控制单元使 PC 内容被放到地址总线 (A_{15} 到 A_8) 和地址 / 数据总线 (AD_7 到 AD_0) 上。在 ALE 脉冲的下降沿，总线上的其他模块存储该地址。

在时序状态 T_2 中，被寻址内存模块把被寻址内存位置的内容放到地址 / 数据总线上。控制单元设置读控制 (RD) 信号来表示读操作，但要等到 T_3 才会从总线复制数据。这就让内存模块有时间把数据放到总线上，并使信号电平稳定下来。最后的 T_4 状态是总线空闲状态，在这个状态中，处理器对指令进行译码。其余的机器周期以类似的方式进行。

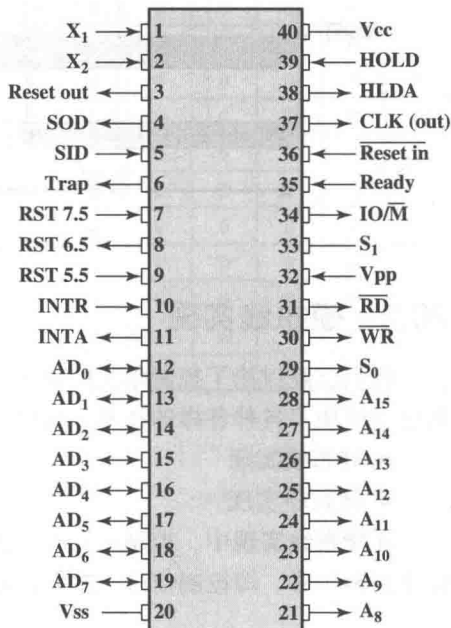


图 20-8 Intel 8085 的引脚配置

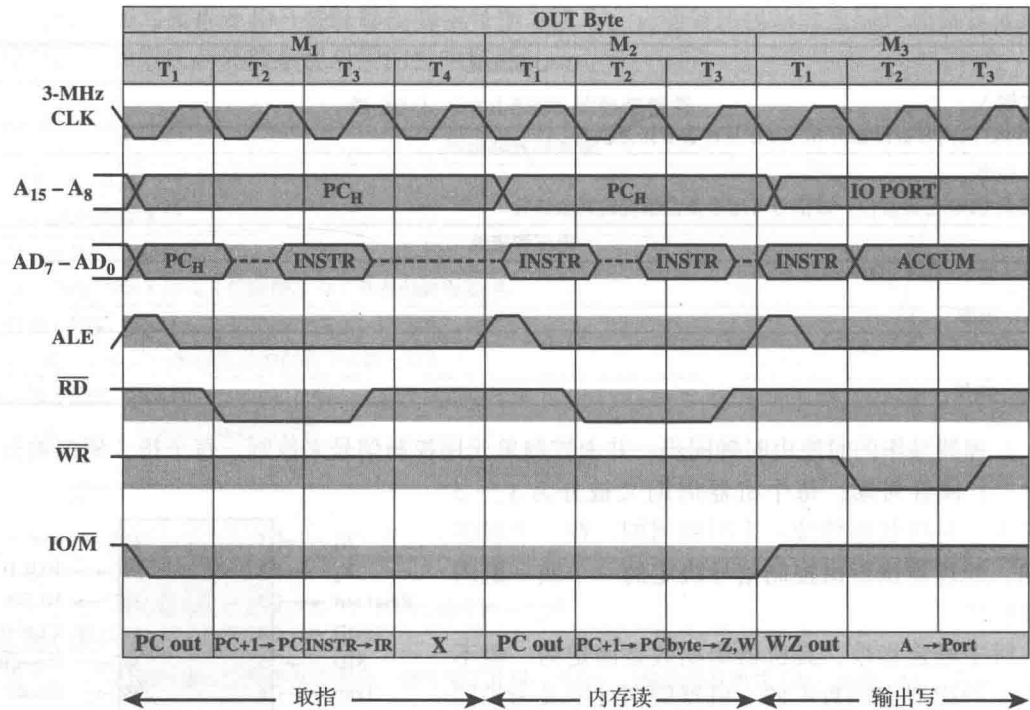


图 20-9 Intel 8085 OUT 指令的时序图

20.3 硬布线实现

我们已经讨论了控制单元的输入、输出和功能。现在要转向控制单元的实现。对这个问题已经使用了各种各样的技术，其中大部分可以分为两类：

- 硬布线实现
- 微程序实现

在硬布线实现中，控制单元本质上是一个状态机电路，它的输入逻辑信号被转换成一组输出逻辑信号，即控制信号。本节将讨论这种方法。微程序实现是第 21 章讨论的主题。

20.3.1 控制单元输入

图 20-4 描绘了迄今为止讨论的控制单元。关键输入是 IR、时钟、标志和控制总线信号。对于标志和控制总线信号，每个单独位通常都具有某种含义（如：溢出）。但是，其他两个输入对于控制单元没有直接作用。

首先考虑 IR。控制单元使用操作码，对不同的指令执行不同的操作（发出不同控制信号的组合）。为了简化控制单元的逻辑，每个操作码应该有唯一的逻辑输入。这个功能可以由译码器来执行，它接收编码的输入，产生一个输出。通常，译码器有 n 个二进制输入和 2^n 个二进制输出。 2^n 个不同输入模式中的每一个都会激活一个唯一的输出。表 20-3 是 $n=4$ 的一个例子。一般，控制单元的译码器要比这复杂得多，以便适应各种长度的操作码。第 11 章给出了一个用于实现译码器的数字逻辑的例子。

控制单元的时钟部分发出重复的脉冲序列。这对度量微操作持续时间很有用。本质上，时钟脉冲的周期必须足够长，以允许信号沿着数据通路传播并穿过处理器电路。但是，正如

我们所看到的，控制单元在一个指令周期的不同时间单位中发出不同的控制信号。所以，我们希望有一个计数器作为控制单元的输入，使不同的控制信号用于 T_1 、 T_2 等。在指令周期结束时，控制单元必须反馈到计数器，以便在 T_1 重新初始化。

表 20-3 4 输入 16 输出的译码器

I1	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
I2	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
I3	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
I4	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
O1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
O2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
O3	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
O4	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
O5	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
O6	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
O7	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
O8	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
O9	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
O10	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
O11	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
O12	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
O13	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
O14	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
O15	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
O16	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

通过这两点改进，控制单元可以如图 20-10 所示。

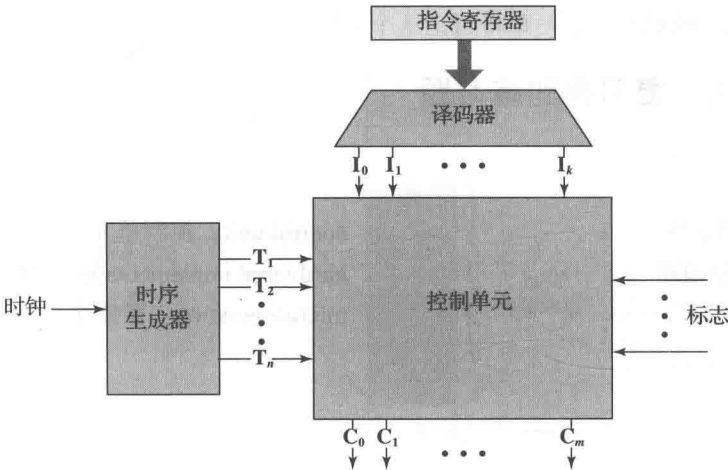


图 20-10 带有译码输入的控制单元

20.3.2 控制单元逻辑

为了定义控制单元的硬布线实现，剩下的就是讨论控制单元的内部逻辑，该逻辑产生输

出控制信号作为其输入信号的函数。

本质上，对于每个控制信号必须要做的是，推导出信号的布尔表达式作为输入的函数。这一点最好用例子来解释。让我们再次考虑如图 20-5 所示的简单例子。我们在表 20-1 中看到了指令周期四个阶段中的三个所需要的微操作序列和控制信号。

让我们考虑一个控制信号 C_5 。这个信号使得数据从外部数据总线读入 MBR。我们可以看到，它在表 20-1 中使用了两次。让我们定义两个新的控制信号：P 和 Q，对它们的解释如下：

PQ=00	取指周期
PQ=01	间接周期
PQ=10	执行周期
PQ=11	中断周期

那么，定义 C_5 的布尔表达式如下：

$$C_5 = \overline{P} \cdot \overline{Q} \cdot T_2 + \overline{P} \cdot Q \cdot T_2$$

也就是说，控制信号 C_5 将在取指周期和间接周期的第二个时间单位声明。

这个表达式不完整。执行周期也需要 C_5 。对于我们的简单例子，让我们假设只有三条指令要从内存读取：LDA、ADD 和 AND。现在可以把 C_5 定义为

$$C_5 = \overline{P} \cdot \overline{Q} \cdot T_2 + \overline{P} \cdot Q \cdot T_2 + P \cdot \overline{Q} \cdot (LDA + ADD + AND) \cdot T_2$$

对于处理器生成的每个控制信号都可以重复这个过程。结果是一组布尔方程式，它定义了控制单元的行为，从而也定义了处理器的行为。

为了把所有的东西都连在一起，控制单元必须控制指令周期的状态。如前所述，在每个子周期（取指、间接、执行、中断）结束时，控制单元发出信号，使得时序生成器重新初始化并发出 T_1 。控制单元还必须设置 P 和 Q 的适当值，以定义下一个要执行的子周期。

读者应该能够理解：在现代复杂处理器中，定义控制单元所需的布尔方程的数量是非常大的。实现满足所有这些方程式的组合电路的任务变得极其困难。结果通常是使用一种简单得多的方法，称为微程序。下一章将讨论这个主题。

20.4 关键词、复习题和练习题

关键词

control bus: 控制总线	control unit: 控制单元
control path: 控制通路	hardwired implementation: 硬布线实现
control signal: 控制信号	micro-operation: 微操作

复习题

- 20.1 请解释指令的书写序列与时间序列之间的区别。
- 20.2 指令与微操作之间有什么关系？
- 20.3 处理器控制单元的总体功能是什么？
- 20.4 请概述作为控制单元特性的三步过程。
- 20.5 控制单元执行的基本任务有哪些？
- 20.6 提供控制单元输入和输出的典型列表。

- 20.7 请列出三种类型的控制信号。
- 20.8 简要解释控制单元硬布线实现的含义。

练习题

- 20.1 你的 ALU 可以把其两个输入寄存器相加，并可以对任一输入寄存器的位进行逻辑求补，但它不能执行减法。数值以二进制补码形式存储。列出你的控制单元执行减法所必需的微操作。
- 20.2 对于图 20-5 所示处理器，以和表 20-1 相同的形式，给出下列指令的微操作和控制信号：
 - 加载累加器
 - 存储累加器
 - 加到累加器
 - AND 到累加器
 - 无条件跳转
 - 如果 AC=0 则跳转
 - 累加器求补
- 20.3 假设沿总线传播和通过图 20-6 中 ALU 的延迟分别为 20 ns 和 100 ns。寄存器从总线复制一个数据所需时间为 10 ns。那么，必须留出多少时间给：
 - a. 数据从一个寄存器到另一个寄存器？
 - b. 程序计数器？
- 20.4 写出如图 20-6 所示总线结构在把一个数值加到 AC 时所需的微操作序列，其中，数值为：
 - a. 立即操作数
 - b. 直接地址操作数
 - c. 间接地址操作数
- 20.5 如图 20-11 实现一个栈（栈的讨论请参见附录 I）。给出微操作序列：
 - a. 出栈
 - b. 栈

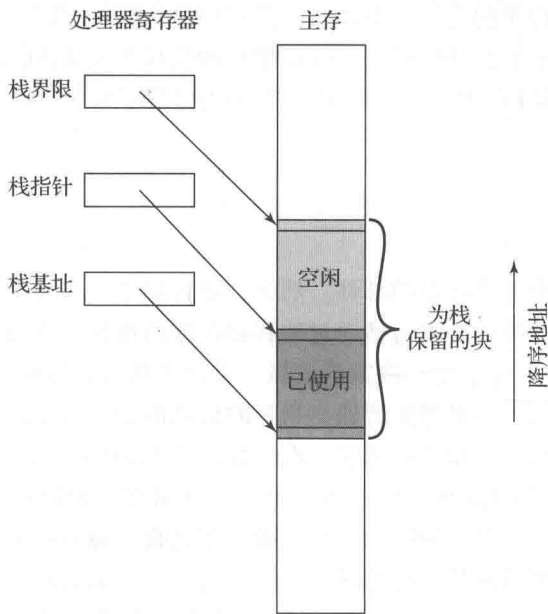


图 20-11 典型栈结构（完整 / 降序）

微程序控制

学习目标

学习本章后，你将可以：

- 概括叙述微程序控制的基本概念。
- 理解硬连线控制和微程序控制之间的差异。
- 讨论排序技术的基本分类。
- 概括叙述微指令的分类。

微程序这个术语最早是在 20 世纪 50 年代早期由 M.V.Wilkes 提出的 [WILK51]。Wilkes 提出了一种控制单元的设计方法，它是有组织的、系统的，并且避免了硬连线实现的复杂性。这一想法激起了许多研究人员的兴趣，但似乎行不通，因为它需要一个快速且相对便宜的控制存储器。

Datamation 在其 1964 年 2 月刊上对微程序技术的现状进行了评论。当时微程序系统还没有广泛使用，其中的一篇文章 [HILL64] 总结了当时流行的观点，即微程序的未来“有些不明朗。没有一家主流制造商对这一技术表现出兴趣，尽管他们大概都对其进行过研究”。

这种情况在短短几个月内就发生了戏剧性的变化。IBM 的 System/360 于 4 月发布，除了最大的型号外，其他所有型号都采用了微程序。尽管 360 系列的出现早于半导体 ROM，但微程序的优势足以让 IBM 做出这一举动。微程序成为 CICS 处理器实现控制单元的一种主流技术。近年来，微程序的使用越来越少，但它仍是计算机设计人员可以使用的工具。例如，正如我们在 Pentium 4 上所看到的，机器指令被转换为类 RISC 格式，其中的大多数不需要使用微程序就可以执行。但是，有些指令是使用微程序来执行的。

21.1 基本概念

21.1.1 微指令

控制单元似乎是一个相当简单的部件。然而，把控制单元实现为基本逻辑单元之间的互连并不是件容易的任务。设计必须包括通过微程序排序的逻辑、执行微操作的逻辑、解释操作码的逻辑以及根据 ALU 标志进行决策的逻辑。设计和测试这样一个硬件是困难的。此外，这种设计相对呆板。例如，如果想要增加一条新的机器指令，那么设计就很难更改。

实现微程序控制单元是一种替代方法，已经在许多 CISC 处理器中得到使用。

考虑表 21-1。除了使用控制信号，每个微操作都用符号来描述。这种表示法看起来很像一种编程语言。实际上它是一种语言，称为**微程序语言**。每一行描述一组同时发生的微操作，称为**微指令**。指令序列被称为**微程序**或**固件**。后一个术语反映了一个事实，即微程序介于硬件和软件之间。用固件设计比用硬件更容易，但写固件程序比写软件程序更难。

表 21-1 Wilkes 示例的机器指令集

规则 / 命令	规则 / 命令的效果
An	$C(\text{Acc}) + C(n)$ 送到 Acc_1
Sn	$C(\text{Acc}) - C(n)$ 送到 Acc_1
Hn	$C(n)$ 送到 Acc_2
Vn	当 $C(n) \geq 0$ 时, $C(\text{Acc}_2) * C(n)$ 送到 Acc
Tn	$C(\text{Acc}_1)$ 送到 n , 0 送到 Acc
Un	$C(\text{Acc}_1)$ 送到 n
Rn	$C(\text{Acc}) * 2^{(n+1)}$ 送到 Acc
Ln	$C(\text{Acc}) * 2^{n+1}$ 送到 Acc
Gn	如果 $C(\text{Acc}) < 0$, 控制转移到 n ; 如果 $C(\text{Acc}) \geq 0$, 则忽略 (即顺序进行)
In	把输入机制上的下一个字符读取到 n
On	把 $C(n)$ 送到输出机制

注: Acc =累加器
 Acc_1 =累加器的高半部分
 Acc_2 =累加器的低半部分
 n =存储位置
 $C(X)$ = X 的内容 (X =寄存器或存储位置)

如何使用微程序的概念来实现一个控制单元呢? 考虑对于每个微操作, 控制单元所能做的就是生成一组控制信号。因此, 对任何微操作来说, 从控制单元发出的每条控制线要么是开, 要么是关。当然, 这种情况可以用每条控制线的二进制数字来表示。所以, 我们可以构造一个控制字, 其中的每一位都代表一条控制线。那么, 每个微操作就可以表示为控制字中不同的 1 和 0 的模式。

假设我们把一串控制字连在一起表示控制单元执行的微操作序列。那么, 接下来, 我们就必须认识到这个微操作顺序不是固定的。有时会有一个间接周期, 有时没有。所以, 让我们把控制字放到一个存储器中, 每个控制字都有唯一的地址。现在, 在每个控制字中增加一个地址字段, 表示当某个条件为真时 (如, 内存引用指令的间接位为 1), 下一个要执行的控制字的位置。另外, 添加一些位来指定条件。

其结果被称为**水平微指令**, 图 21-1a 给出了一个例子。微指令或控制字的格式如下。每个内部处理器控制线都有一位, 每个系统总线的控制线也都有一位。有一个条件字段, 表示应该在什么条件下分支; 还有一个字段表示在分支成立时, 下一条要执行的微指令的地址。这种微指令解释如下:

- 1) 要执行这条微指令, 打开所有为 1 的控制线, 中断所有为 0 的控制线。产生的控制信号将使一个或多个微操作执行。
- 2) 如果由条件位指示的条件为假, 则顺序执行下一条微指令。
- 3) 如果由条件位指示的条件为真, 则由地址字段指示要执行的下一条微指令。

图 21-2 显示了如何在**控制存储器**中安排这些控制字或微指令。每个例程中的微指令要按顺序执行。每个例程都以一个分支或跳转指令结束, 以指示下一步要去哪里。有一个特殊的执行周期例程, 其唯一目的是表示接下来将要执行的一个机器指令例程 (AND、ADD 等), 具体取决于当前的操作码。

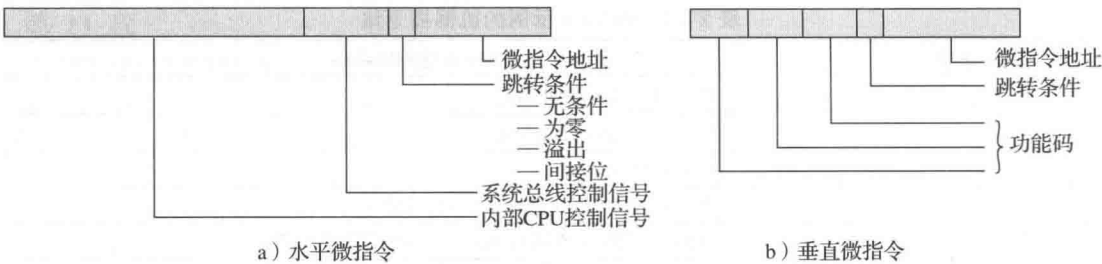


图 21-1 典型微指令格式

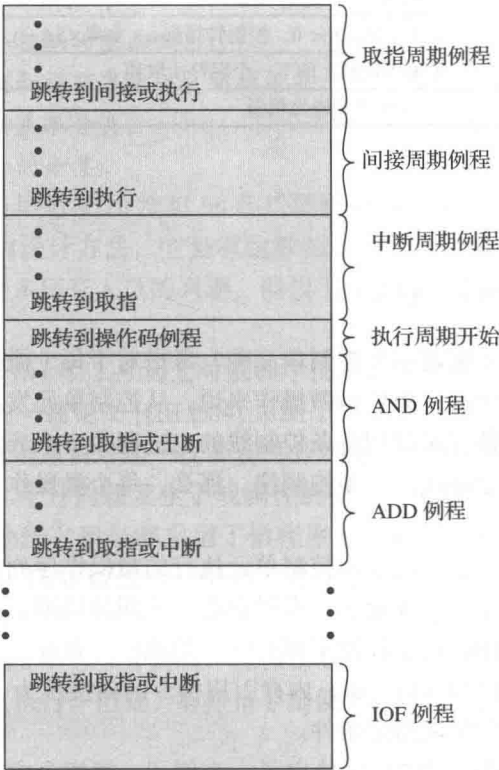


图 21-2 控制存储器的组织结构

图 21-2 的控制存储器是对控制单元完整操作的简明描述。它定义了每个周期（取指、间接、执行、中断）执行的微操作序列，并指定了这些周期的顺序。如果没有其他的，此表示法将是记录特定计算机控制单元功能的有用设备。但还不止于此。这也是实现控制单元的一种方法。

21.1.2 微程序控制单元

图 21-2 的控制存储器包含描述控制单元行为的程序。因此，我们只需要执行该程序就可以实现控制单元。

图 21-3 显示了这种实现的关键要素。微指令集存储在控制存储器中。控制地址寄存器包含下一条要读取的微指令的地址。当一条微指令从控制存储器中读出时，它会被传送到控制缓冲寄存器。该寄存器左边部分（参见图 21-1a）连接来自控制单元的控制线。所以，

从控制存储器中读取一条微指令与执行该微指令是一样的。图中显示的第三个要素是排序单元，它加载控制地址寄存器并发出读命令。

让我们更细致地研究一下这个结构，如图 21-4 所示。与图 21-3 相比，我们可以看到控制单元仍然具有相同的输入（IR、ALU 标志、时钟）和输出（控制信号）。控制单元的功能如下：

1) 为了执行一条指令，排序逻辑单元向控制存储器发出 READ 命令。

2) 控制地址寄存器指定地址中的字被读进控制缓冲寄存器。

3) 控制缓冲寄存器的内容生成控制信号以及排序逻辑单元的下一个地址信息。

4) 排序逻辑单元根据来自控制缓冲寄存器的下一个地址信息以及 ALU 标志，把新地址加载到控制地址寄存器。

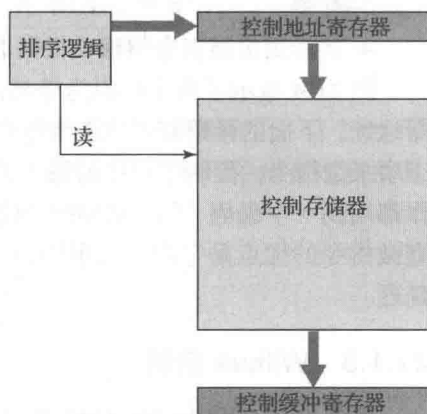


图 21-3 控制单元微架构

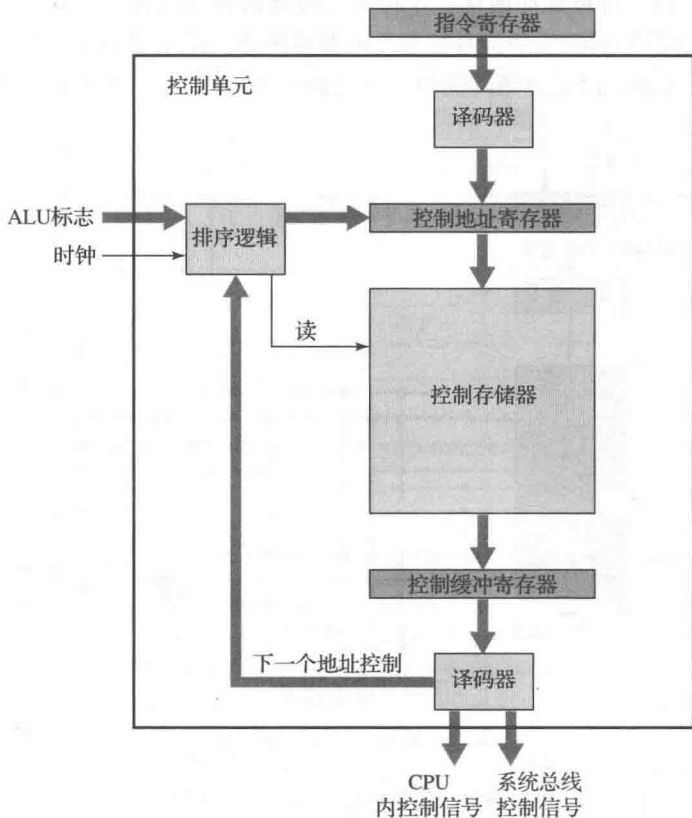


图 21-4 微程序控制单元功能

所有这些都是在一个时钟脉冲中发生的。

刚才列出的最后一步需要详细说明。在每条微指令结束时，排序逻辑单元把新地址加载到控制地址寄存器。根据 ALU 标志的值和控制缓冲寄存器，将会从以下三个决定中选择一个：

- 获取下一条指令 控制地址寄存器加 1。

- **跳转到基于跳转微指令的新例程** 把控制缓冲寄存器的地址字段加载到控制地址寄存器。
- **跳转到机器指令例程** 根据 IR 中的操作码加载控制地址寄存器。

图 21-4 显示了两个标识为译码器的模块。上面的译码器把 IR 的操作码转换成控制存储器地址。下面的译码器不用于水平微指令，而是用于**垂直微指令**（图 21-1b）。如前所述，对于水平微指令，控制字段中的每一位都连到一条控制线上。在垂直微指令中，执行每一个操作都用到一个编码（如： $MAR \leftarrow (PC)$ ），该译码器把这个编码转换成单独的控制信号。垂直微指令的优点是它们比水平微指令更紧凑（位数更少），代价是会增加少量的逻辑和时间延迟。

21.1.3 Wilkes 控制

如前所述，Wilkes 在 1951 年首次提出使用微程序控制单元 [WILK51]。这个提议随后被阐述为更加详细的设计 [WILK53]。研究这一开创性的提议是有意义的。

Wilkes 提出的结构如图 21-5 所示。该系统的核心是部分填充二极管的矩阵。在一个机器周期中，矩阵中的一行被脉冲激活。这将在二极管所在的地方产生信号（图中用一个点表示）。该行的第一部分生成的控制信号控制处理器的操作。第二部分生成在下一个机器周期中将要脉冲激活的行地址。因此，矩阵的每一行就是一条微指令，矩阵的布局就是控制存储器。

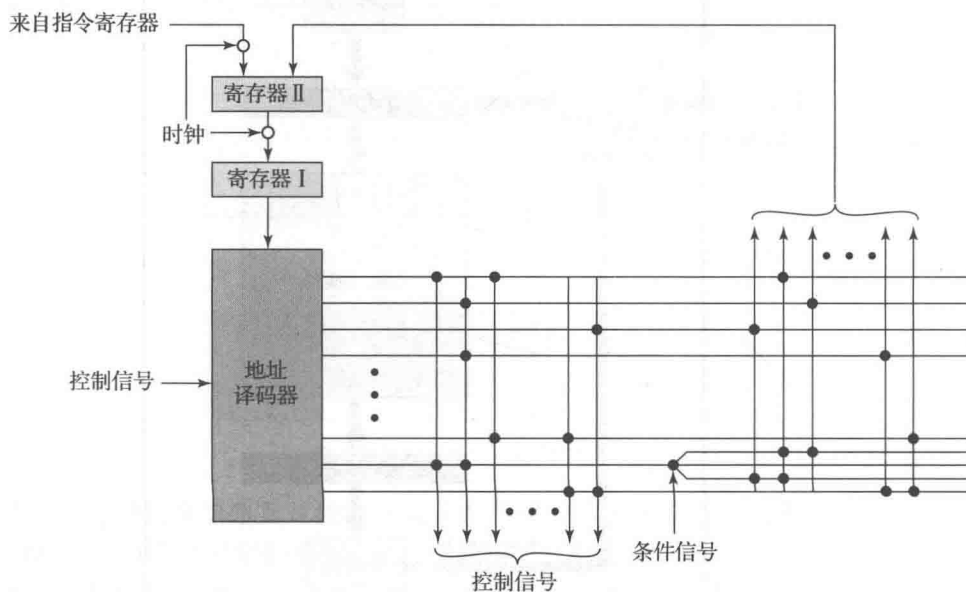


图 21-5 Wilkes 微程序控制单元

在周期开始的时候，将要被脉冲激活的行地址在寄存器 I 中。这个地址是译码器的输入，当被时钟脉冲激活时，它就会激活矩阵的一行。在这个周期中，根据控制信号，指令寄存器中的操作码或者被激活行的第二部分被传递到寄存器 II。然后，寄存器 II 通过时钟脉冲被选通到寄存器 I。交替的时钟脉冲用于激活矩阵的一行，以及把寄存器 II 传输到寄存器 I。由于译码器是一个简单的组合电路，需要双寄存器配置，如果只有一个寄存器，那么在一个周期中，输出将变成输入，从而导致不稳定的情况。

这种方案与前面描述的水平微程序方法（图 21-1a）非常相似。主要的区别是：在前面的描述中，控制地址寄存器会加 1 来得到下一个地址；在 Wilkes 方案中，下一个地址包含在微指令中。为了执行分支，一行必须包含两个地址部分，由条件信号（如标志）来控制，如图 21-5 所示。

在提出该方案后，Wilkes 提供了一个使用它实现简单机器控制单元的例子。这个例子是第一个已知的微程序处理器设计，因为它说明了许多现代微程序原理，所以值得在这里再现。

假设机器（Wilkes 的示例机器）的处理器包含以下寄存器：

A	被乘数
B	累加器（低半部分）
C	累加器（高半部分）
D	移位寄存器

此外，还有三个寄存器和两个 1 位标志仅供控制单元访问。三个寄存器如下：

E	用作内存地址寄存器（MAR）和临时存储器
F	程序计数器
G	另一个暂存寄存器。用于计数

表 21-1 列出了这个例子的机器指令集。表 21-2 是实现控制单元的完整微指令集，用符号形式表示。因此，总共 38 条微指令就是定义整个系统所需的全部。

表 21-2 Wilkes 示例的微指令

		算术单元	控制寄存器单元	条件触发器		下一条微指令	
				设置	使用	0	1
	0		F 到 G 和 E			1	
	1		(G 到 “1”) 到 F			2	
	2		存储到 G			3	
	3		G 到 E			4	
	4		E 到译码器			—	
A	5	C 到 D				16	
S	6	C 到 D				17	
H	7	存储到 B				0	
V	8	存储到 A				27	
T	9	C 到存储				25	
U	10	C 到存储				0	
R	11	B 到 D	E 到 G			19	
L	12	C 到 D	E 到 G			22	
G	13		E 到 G	(1) C_5		18	
I	14	输入到存储				0	
O	15	存储到输出				0	
	16	(D + 存储) 到 C				0	
	17	(D - 存储) 到 C				0	
	18				1	0	1
	19	D 到 $B(R)^{\text{①}}$	($G-1$) 到 E			20	
	20	C 到 D		(1) E_5		21	

(续)

		算术单元	控制寄存器单元	条件触发器		下一条微指令	
				设置	使用	0	1
	21	D 到 $C(R)$			1	11	0
	22	D 到 $C(L)$ ^②	$(G-1)$ 到 E			23	
	23	B 到 D		$(1)E_5$		24	
	24	D 到 $B(L)$			1	12	0
	25	“0” 到 B				26	
	26	B 到 C				0	
	27	“0” 到 C	“18” 到 E			28	
	28	B 到 D	E 到 G	$(1)B_1$		29	
	29	D 到 $B(R)$	$(G-“1”)$ 到 E			30	
	30	C 到 $D(R)$		$(2)E_5$	1	31	32
	31	D 到 C			2	28	33
	32	$(D+A)$ 到 C			2	28	33
	33	B 到 D		$(1)B_1$		34	
	34	D 到 $B(R)$				35	
	35	C 到 $D(R)$			1	36	37
	36	D 到 C				0	
	37	$(D-A)$ 到 C				0	

注： $A, B, C, \dots$ 表示算术和逻辑单元中的各种寄存器。“ C 到 D ”表示开关电路把寄存器 C 的输出连接到寄存器 D 的输入；“ $(D+A)$ 到 C ”表示 A 的输出寄存器连接到加法单元的一个输入（ D 的输出总是连接到另一个输入），加法器的输出连接到寄存器 C 。引号中的数字符号 n （如“ n ”）表示源，其输出是以最低有效数字为单位的数字 n 。

- ① 右移。设置了算术单元中的开关电路，这样在右移微操作中，寄存器 C 的最低有效位就被放在寄存器 B 的最高有效位上，而寄存器 C 的最高有效位（符号位）则重复（从而对负数进行修正）。
- ② 左移。类似设置开关电路，在左移微操作中，把寄存器 B 的最高有效位传递到寄存器 C 的最低有效位。

第1整列给出了每条微指令的地址（行号）。对应这些地址的操作码被标记。因此，当遇到加法指令（ A ）的操作码时，将会执行位置5的微指令。第2列和第3列分别表示ALU和控制单元会采取的操作。每个符号表示都被转换成一组控制信号（微指令位）。第4列和第5列与两个标志（触发器）的设置和使用有关。第4列指明设置标志的信号。例如， $(1)C_5$ 表示由寄存器 C 中数字的符号位来设置标志1。如果第5列包含标志标识符，那么第6列和第7列就包含要使用的两个可选微指令地址。否则，第6列就指明要获取的下一条微指令的地址。

指令0到指令4构成取指周期。微指令4把操作码送给译码器，译码器生成与所取机器指令对应的微指令的地址。读者应该能够通过仔细研究表21-2来推出控制单元的完整功能。

21.1.4 优缺点

使用微程序来实现控制单元的主要优点是简化了控制单元的设计。因此，实现起来成本更低，也更不易出错。硬连线控制单元必然包含复杂逻辑，以通过指令周期的多个微操作进行排序。另一方面，微程序控制单元的译码器和排序逻辑单元也是非常简单的逻辑组件。

微程序单元的主要缺点是它比同类技术的硬连线单元要慢。尽管如此，由于易于实现，

微程序在纯 CISC 架构中仍是实现控制单元的主要技术。RISC 处理器的指令格式更简单，它通常使用硬连线控制单元。我们现在更详细地研究微程序方法。

21.2 微指令排序

微程序控制单元要实现如下两个基本任务：

- **微指令排序** 从控制存储器获取下一条微指令。
- **微指令执行** 生成执行微指令所需的控制信号。

在设计控制单元时，这些任务必须一起考虑，因为它们都会影响微指令的格式和控制单元的时序。本节中，我们将关注排序，并尽可能少地提及格式和时序问题。这些问题将在下一节进行更为细致的讨论。

21.2.1 设计注意事项

微指令排序技术涉及两个问题：微指令的大小和地址生成时间。第一个问题显而易见，最小化控制存储器会降低组件的成本。第二个问题就是想要尽可能快地执行微指令。

执行微程序时，下一条要执行的微指令的地址有如下几种类别：

- 由指令寄存器决定
- 下一个顺序地址
- 分支

第一类只在每个指令周期出现一次，就在取得一条指令之后。第二类是大多数设计中最常见的。但是，设计不能只针对顺序访问来优化。不论是条件分支还是无条件分支，都是微程序的必要部分。此外，微指令序列常常很短，每 3 或 4 条微指令中就有一条可能是分支 [SIEW82]。所以，设计紧凑、高效的微指令分支技术是很重要的。

21.2.2 排序技术

根据当前微指令、条件标志和指令寄存器的内容，必须为下一条微指令生成控制存储器地址。对此，已经使用了各种技术。我们可以把它们分为三大类，如图 21-6 ~ 图 21-8 所示。这些类别是以微指令中的地址信息格式为基础的：

- 双地址字段
- 单地址字段
- 可变格式

最简单的方法就是每条微指令提供两个地址字段。图 21-6 显示了如何使用这些信息。提供了一个多路复用器，作为两个地址字段和指令寄存器的目的地。根

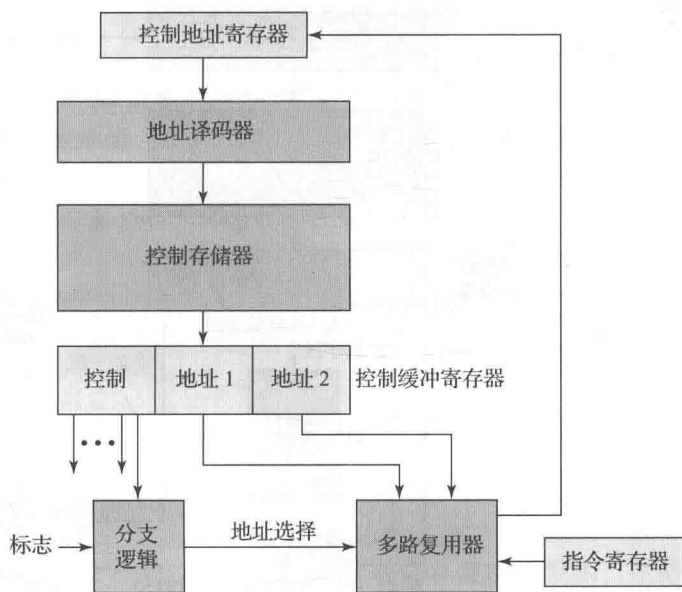


图 21-6 分支控制逻辑：双地址字段

据地址选择输入，复用器把操作码或两个地址中的一个传送到控制地址寄存器（CAR）中。CAR 随后被译码，以产生下一条微指令地址。地址选择信号由分支逻辑模块提供，该模块的输入是控制单元标志以及来自微指令控制部分的位。

虽然双地址方法很简单，但相比其他方法，它在微指令中需要更多的位。通过一些额外的逻辑，可以实现节省。一个常用的方法是使用单地址字段（图 21-7）。使用这种方法，下一个地址的选项如下：

- 地址字段
- 指令寄存器编码
- 下一个顺序地址

由地址选择信号决定应

该选择哪个选项。这个方法把地址字段的数量减少到一个。但是要注意的是，这个地址字段通常不会被使用。因此，微指令编码方案的效率有点低。

另一种方法是提供两种完全不同的微指令格式（图 21-8）。一位表示正在使用的格式。在一种格式中，其余的位被用于激活控制信号。在另一种格式中，一些位驱动分支逻辑模块，其余位提供地址。对于第一种格式，下一个地址要么是下一个顺序地址，要么是从指令寄存器派生的地址。对于第二种格式，可以指定条件分支或无条件分支。这种方法的一个缺点是每个分支微指令都会占用一个完整的周期。对于其他方法，地址生成与控制信号生成在同一个周期中，从而最大限度地减少了控制存储器的访问。

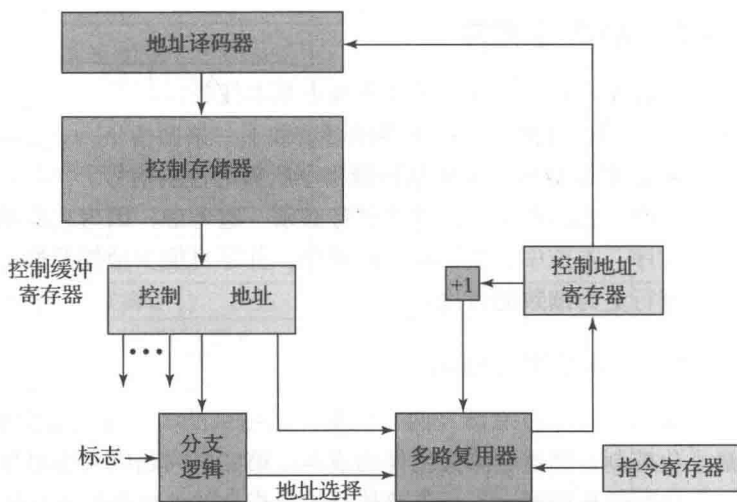


图 21-7 分支控制逻辑：单地址字段

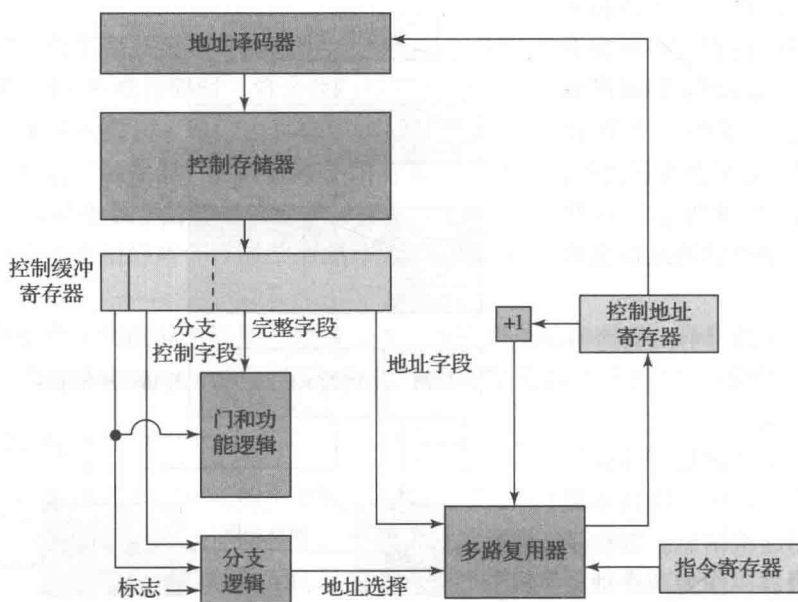


图 21-8 分支控制逻辑：可变格式

刚刚描述的方法都是通用的。具体实现一般会涉及这些技术的变体或组合。

21.2.3 地址生成

我们已经从格式考量和通用逻辑需求的角度研究了排序问题。另一种观点是考虑派生或计算下一个地址的各种方法。

表 21-3 列出了各种地址生成技术。这些技术可以分为显式技术和隐式技术：显式技术的地址由微指令显式提供，隐式技术需要额外的逻辑来生成地址。

表 21-3 微指令地址生成技术

显式	隐式
双字段	映射
无条件分支	加法
条件分支	后效控制

实际上我们已经讨论了显式技术。对于双字段方法，每条微指令有两个可选地址。使用单地址字段或可变格式，可以实现各种分支指令。条件分支指令取决于下列类型的信息：

- ALU 标志。
- 机器指令的操作码部分或地址模式字段。
- 被选寄存器的部分，如符号位。
- 控制单元中的状态位。

通常也会使用一些隐式技术。映射就是其中一种，几乎所有的设计都需要它。机器指令的操作码部分必须映射到微指令地址。这在每个指令周期中只发生一次。

一种常见的隐式技术是把地址的两个部分进行组合或相加，以形成完整的地址。这个方法被用于 IBM S/360 系列 [TUCK67]，并在许多 S/370 型号上使用。我们将以 IBM 3033 为例。

IBM 3033 的控制地址寄存器有 13 位长，如图 21-9 所示。可以区分地址的两个部分。从一个微指令周期到下一个微指令周期时，最高的 8 位（00 ~ 07）通常不会改变。在微指令执行的过程中，这 8 位直接从微指令的 8 位字段（BA 字段）复制到控制地址寄存器的最高 8 位上。它在控制存储器中定义了一个 32 条微指令的块。控制地址寄存器的其余 5 位被设置为指定接下来要获取的微指令的特定地址。这些位中的每一个都由当前微指令中的一个 4 位字段（除了一个是 7 位字段）来决定，该字段指定设置相应位的条件。例如，根据最近 ALU 操作是否发生了进位，控制地址寄存器中的位可能被设置为 1 或 0。

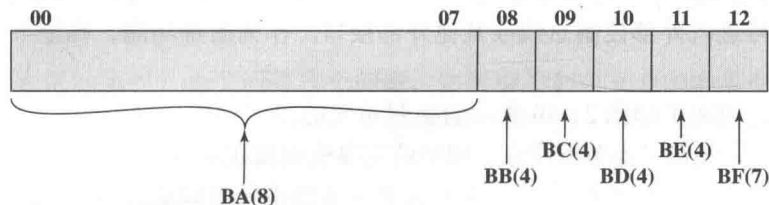


图 21-9 IBM 3033 控制地址寄存器

表 21-3 列出的最后一种方法称为后效控制。这种方法涉及微指令地址的使用，该微指令地址之前被保存在控制单元中的临时存储器中。例如，某些微指令集配备了子例程功

能。内部寄存器或寄存器栈被用于保存返回地址。LSI-11 使用了这个方法，我们现在来研究一下。

21.2.4 LSI-11 微指令排序

LSI-11 是 PDP-11 的微型计算机版本，系统的主要组件都在一块单板上。LSI-11 是用微程序控制单元实现的 [SEBE76]。

LSI-11 使用了 22 位微指令和 2 K 个 22 位字的控制存储器。确定下一条微指令的地址有 5 种方法：

- **下一条顺序地址** 在没有其他指令的情况下，控制单元的控制地址寄存器加 1。
- **操作码映射** 在每个指令周期开始的时候，由操作码决定下一条微指令的地址。
- **子例程功能** 一会儿解释。
- **中断测试** 某些微指令指定了中断测试。如果发生中断，这将决定下一条微指令的地址。
- **分支** 使用条件和无条件分支微指令。

这里提供了一层子例程功能。每条微指令中有一位专门用于此任务。当该位被置位时，把控制地址寄存器中的已更新内容加载到一个 11 位的返回寄存器。指定返回的后续微指令将使得控制地址寄存器从返回寄存器加载内容。

返回是无条件分支指令的一种形式。无条件分支的另一种形式将使得控制地址寄存器从微指令中的 11 位来加载位。条件分支指令使用微指令中的一个 4 位测试码。这个测试码指定对各种 ALU 条件码进行测试，以确定分支决策。如果条件不为真，就选择下一个顺序地址。如果条件为真，将把微指令的 8 位加载到控制地址寄存器的低 8 位。这允许在 256 字的内存页中进行分支。

可以看到，LSI-11 在控制单元中包含了一个强大的地址排序功能。这使得微程序员具有相当大的灵活性，并能简化微程序任务。另一方面，与更简单的功能相比，这个方法需要更多的控制单元逻辑。

21.3 微指令执行

微指令周期是微程序处理器的基本事件。每个周期都有两个组成部分：取指和执行。取指部分由微指令地址的生成决定，这在前一节中已经介绍过了。本节将讨论微指令的执行。

回忆一下，微指令执行的结果是产生控制信号。其中一些信号控制点在处理器内部。其余的信号进入外部控制总线或其他外部接口。作为附带功能，确定下一条微指令的地址。

前面的描述说明了如图 21-10 所示的控制单元的组织结构。它对图 21-4 所示的那一版进行了少许修订，强调了本节的重点。图中的主要模块现在应该很清楚了。排序逻辑模块包含的逻辑能执行上一节讨论的功能。它生成下一条微指令的地址，用作指令寄存器、ALU 标志、控制地址寄存器（递增）和控制缓冲寄存器的输入。最后一个可以提供实际地址、控制位，或者两者皆能提供。该模块由时钟驱动，时钟决定了微指令周期的时序。

控制逻辑模块生成控制信号作为微指令位的函数。显然，微指令的格式和内容决定了控制逻辑模块的复杂性。

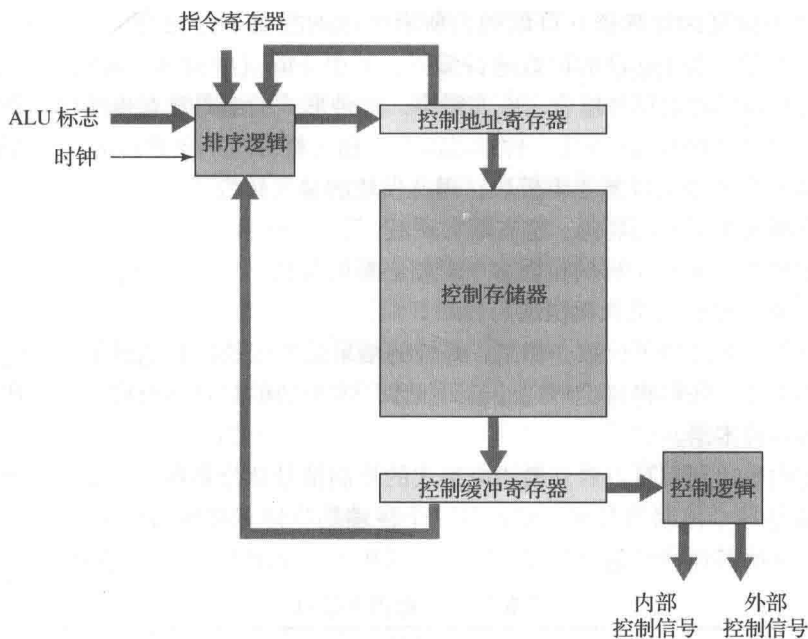


图 21-10 控制单元的组织结构

21.3.1 微指令分类法

微指令可以按各种方法来分类。在文献中常见的有如下几种：

- 垂直 / 水平
- 压缩 / 非压缩
- 硬 / 软微编程
- 直接 / 间接编码

所有这些都与微指令的格式有关。在文献中，这些术语都没有以一致的、精确的方式使用过。但是，对这些特质对的检查有助于阐明微指令设计的备选方案。在接下来的段落中，我们首先来看看这些特质对背后的关键设计问题，然后再探讨每一对特质所建议的概念。

在 Wilkes 最初的提议中 [WILK51]，微指令中的每一位要么直接产生一个控制信号，要么直接产生下一个地址中的一位。在前一节中，我们已经看到了，使用更少的微指令位可以实现更复杂的地址排序方案。这些方案需要更复杂的排序逻辑模块。与控制信号相关的微指令部分也存在类似的权衡。通过对控制信息进行先编码再译码来产生控制信号，可以节省控制字的位。

如何进行这样的编码？为了回答这个问题，要考虑控制单元总共需要驱动 K 个不同的内部和外部控制信号。在 Wilkes 方案中， K 位微指令专门用于这一目的。这就使得在任一指令周期中，可以生成 2^K 种可能的控制信号组合。但是，如果观察到并非所有可能的组合都会被使用，那么我们将会做得更好。示例如下：

- 两个源不能被选通到同一个目的地（如图 21-5 中的 C_2 和 C_8 ）。
- 一个寄存器不能既是源又是目的（如图 21-5 中的 C_5 和 C_{12} ）。
- 一次只能向 ALU 提供一种模式的控制信号。
- 一次只能向外部控制总线提供一种模式的控制信号。

因此，对于给定的处理器，可以列出所有可能的控制信号组合，从而有 $Q < 2^K$ 种可能性。它们可以用最少为 $\log_2 Q$ 的位数进行编码，其中 $(\log_2 Q) < K$ 。这将是最紧密的编码，可以保留所有允许的控制信号组合。在实践中，这种形式的编码没有被使用，原因有两个：

- 它和纯译码（Wilkes）方案一样难以编程。稍后将对这一点进行进一步的讨论。
- 它需要一个复杂的控制逻辑模块，因此模块的速度较慢。

与之相反的是做了一些妥协。这有两种类型：

- 对可能的组合进行编码的位数多于严格必要的位数。
- 某些物理上允许的组合无法编码。

后一种妥协的效果是减少了位数。但是，最终的结果是使用的位数超过了 $\log_2 Q$ 。

在下一小节中，我们将讨论特定的编码技术。本小节的其余部分将讨论编码的影响以及用于描述它的各种术语。

基于前述内容，我们可以看到微指令格式的控制信号部分是在一个范围之内的。这个范围的一个极端是每个控制信号有一位；另一个极端则是使用高度编码格式。表 21-4 显示了微程序控制单元的其他特性也有范围，这些范围在很大程度上取决于编码程度的范围。

表 21-4 微指令范围

特性	
无编码	高度编码
许多位	几位
详细的硬件视图	大致的硬件视图
难于编程	易于编程
充分利用并行性	未充分利用并行性
少量或无控制逻辑	复杂控制逻辑
执行快	执行慢
优化性能	优化编程
术语	
非压缩	压缩
水平	垂直
硬	软

表 21-4 中第二项的对比相当明显。纯粹的 Wilkes 方案将需要最多的位数。还应明确指出，这种极端呈现了硬件最详细的视图。每个控制信号都是由微程序员单独控制。编码的方式是汇聚功能或资源，这样微程序员就可以在更高、更粗略的层次上查看处理器。此外，编码的目的是减轻微编程的负担。同样，应该清楚的是，理解和协调全部控制信号的使用是一项困难的任务。如前所述，编码的后果之一通常是防止使用某些其他允许的组合。

上一段从微程序员的角度讨论了微指令的设计。但编码度还可以从其硬件效果来观察。对于纯的未编码格式，很少或不需要译码逻辑；每一位都产生一个特定的控制信号。当使用更紧凑、更聚合的编码方案时，就需要更复杂的译码逻辑。反过来，这又有可能影响性能。通过更复杂的控制逻辑模块的门来传递控制信号需要更多的时间。因此，编码微指令的执行比未编码微指令的执行需要更长的时间。

因此，表 21-4 列出的所有特性都属于设计策略的范围。通常，位于这个范围左端的设计旨在优化控制单元的性能。位于右端的设计更关注于优化微编程的过程。实际上，靠近该范围右端的微指令集看起来很像机器指令集。这个方面一个很好的例子是 LSI-11 设计，本

节稍后将对此进行讨论。通常，当目标仅仅是实现一个控制单元时，设计将接近该范围的左端。目前讨论的 IBM 3033 设计就属于这一类。正如我们稍后将讨论的那样，一些系统允许各种用户使用相同的微指令功能来构建不同的微程序。在后一种情况下，设计可能接近于范围的右端。

我们现在可以处理前面介绍的一些术语了。表 21-4 给出了其中三对术语是如何与微指令范围相关的。本质上，所有这些术语对都描述了相同的东西，但却强调了不同的设计特性。

压缩的程度与给定控制任务和特定微指令位之间的识别度有关。当位变得更加紧凑时，给定位数就能包含更多信息。因此，压缩意味着编码。术语水平和垂直与微指令相对宽度有关。[SIEW82] 建议，根据经验，垂直微指令的长度在 16 ~ 40 位之间，水平微指令的长度在 40 ~ 100 位之间。术语硬和软微编程用来表示与底层控制信号和硬件布局之间的密切程度。硬微程序通常是固定的，且提交到只读存储器。软微程序更具可变性，且提示用户进行微编程。

本小节开始提到的另一对术语是指直接编码和间接编码，现在来讨论这个主题。

21.3.2 微指令编码

实际上，微编程控制单元的设计没有使用纯无编码或水平微指令格式。至少要使用某种程度的编码来减少控制存储器宽度，并简化微程序设计任务。

编码的基本技术如图 21-11a 所示。微指令被组织成一组字段。每个字段包含一个编码，其译码后会激活一个或多个控制信号。

让我们考虑一下这种布局的含义。当执行微指令时，每个字段被译码并生成控制信号。所以， N 个字段就指定了 N 个同步操作。每个动作都会激活一个或多个控制信号。通常，但并不总是，我们想要设计这样一种格式，即每个控制信号被不超过一个的字段激活。但是，显然，每个控制信号都必须至少被一个字段激活。

现在考虑单个字段。一个字段由 L 位构成，它可以包含 2^L 个编码中的一个，其中的每一个都可以编码为不同的控制信号模式。因为，一次只能有一个编码出现在字段中，所以这些编码是互斥的，由此，它们所导致的操作也是互斥的。

现在可以用简单的术语来说明编码微指令格式的设计：

- 将格式组织成单独的字段。也就是说，每个字段描述一组动作（控制信号的模式），这样不同字段的动作可以同时发生。
- 定义每个字段，使得该字段指定的候选动作是互斥的。也就是说，一次只能为给定字段指定一个操作。

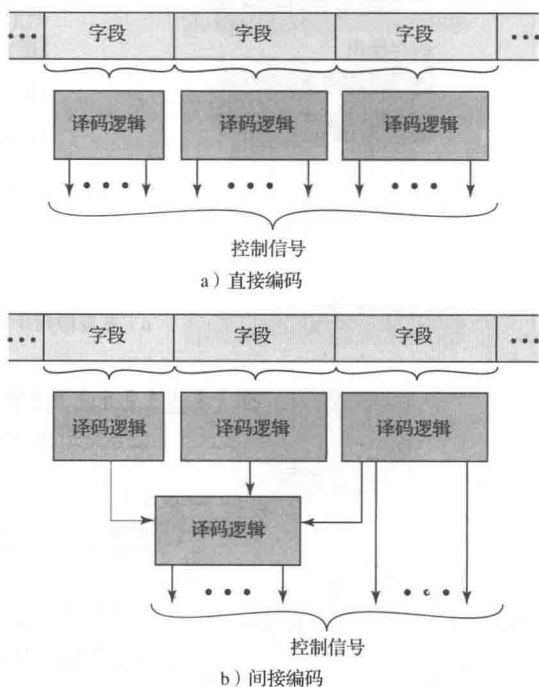
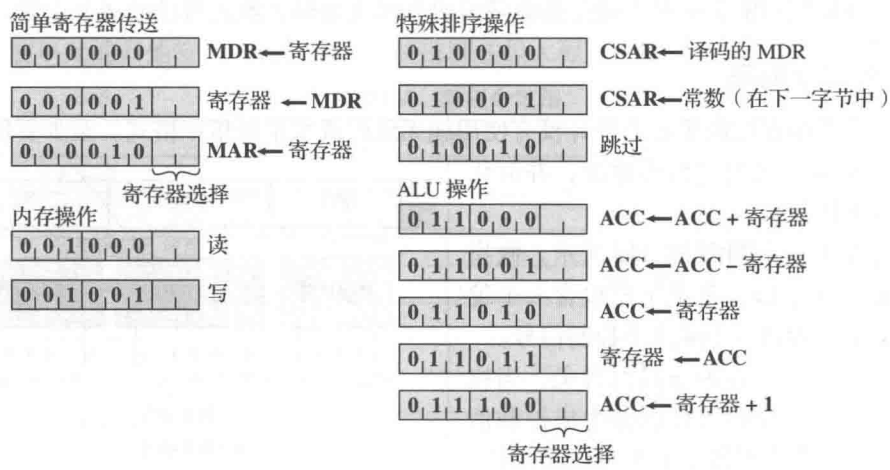


图 21-11 微指令编码

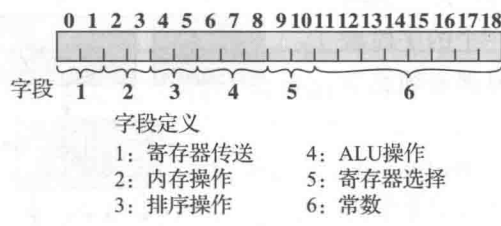
有两种方法可以把编码后的微指令组织成字段：功能型和资源型。功能型编码方法标识机器内的功能，并根据功能类型指定字段。例如，如果可以使用各种源把数据传递给累加器，那么就可以为此指定一个字段，其中的每个编码指定一个不同的源。资源型编码把机器看作由一组独立资源组成，每个资源（如 I/O、内存、ALU）占用一个字段。

编码的另一个方面是：它是直接编码，还是间接编码（图 21-11b）。对于间接编码，一个字段用于确定另一个字段的解释。例如，考虑一个 ALU 能执行 8 种不同的算术运算和 8 种不同的移位操作。一个 1 位的字段可用于指示使用的是移位操作还是算术运算；一个 3 位的字段表示操作。这种技术通常意味着两级译码，增加了传播延迟。

图 21-12 是这些概念的一个简单示例。假设处理器有一个累加器和几个内部寄存器，比如程序计数器和用于 ALU 输入的暂存寄存器。图 21-12a 显示了一个高度垂直的格式。前 3 位表示操作类型，接下来的 3 位对操作进行编码，最后 2 位选择一个内部寄存器。图 21-12b 是更为水平的方法，但仍然使用了编码。本例中，不同功能出现在不同的字段中。



a) 垂直微指令格式



b) 水平微指令格式

图 21-12 简单机器的两种微指令格式

21.3.3 LSI-11 微指令执行

LSI-11[SEBE76] 是垂直微指令方法的一个很好的例子。我们首先来看看控制单元的组织结构，然后再来看微指令格式。

LSI-11 控制单元组织结构 LSI-11 是 PDP-11 系列中的第一成员，它提供了单板处理器。单板包含了 3 个 LSI 芯片，被称为微指令总线（MIB）的内部总线和一些额外的接口逻辑。

图 21-13 用简化的形式绘制了 LSI-11 处理器的组织结构。三个芯片分别是数据芯片、

控制芯片和控制存储芯片。数据芯片包含一个 8 位的 ALU、26 个 8 位的寄存器以及多个条件码的存储器。其中 16 个寄存器用于实现 PDP-11 的 8 个 16 位通用寄存器。其他包括程序状态字、内存地址寄存器 (MAR) 和内存缓冲寄存器。由于 ALU 一次只能处理 8 位, 要实现 16 位 PDP-11 算术运算就需要通过 ALU 两次。这是由微程序控制的。

控制存储芯片或芯片组包含 22 位宽的控制存储器。控制芯片包含排序和微指令执行逻辑。它含有控制地址寄存器, 控制数据寄存器和机器指令寄存器的副本。

MIB 把所有的组件连在一起。在微指令取指期间, 控制芯片在 MIB 上产生一个 11 位的地址。控制存储被访问, 生成一个 22 位的微指令, 并把它放到 MIB 上。低 16 位进入数据芯片, 低 18 位进入控制芯片。高 4 位控制特殊处理器板功能。

图 21-14 提供了 LSI-11 控制单元的视图, 虽然仍然是简化的, 但要更详细一些: 图中省略了单个芯片边界。在两个模块中实现了 21.2 节中描述的地址排序方案。整体顺序控制由微程序顺序控制模块提供, 该模块能够增加微指令地址寄存器, 并执行无条件分支。其他形式的地址计算由单独的转换数组来完成。它是一个组合电路, 根据微指令、机器指令、微指令程序计数器和中断寄存器来生成地址。

转换数组在下列情况下的作用:

- 操作码用于决定微例程的开始。
- 在适当的时间, 测试微指令地址模式位以执行合适的寻址模式。
- 定期测试中断条件。
- 评估条件分支微指令。

LSI-11 微指令格式 LSI-11 使用非常垂直的微指令格式, 它只有 22 位宽。微指令组与

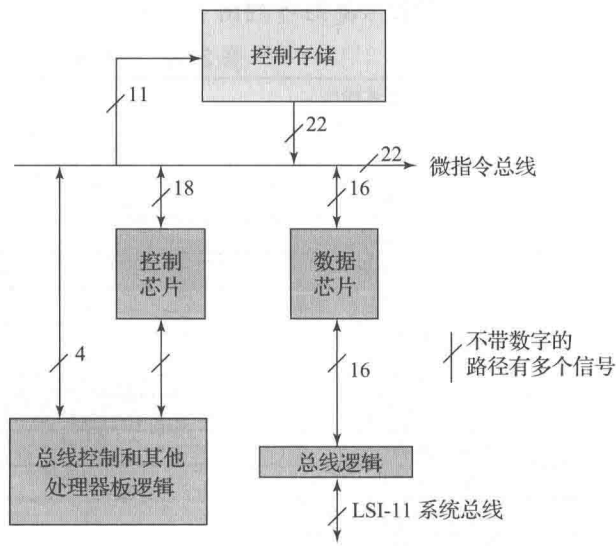


图 21-13 LSI-11 处理器的简化方框图

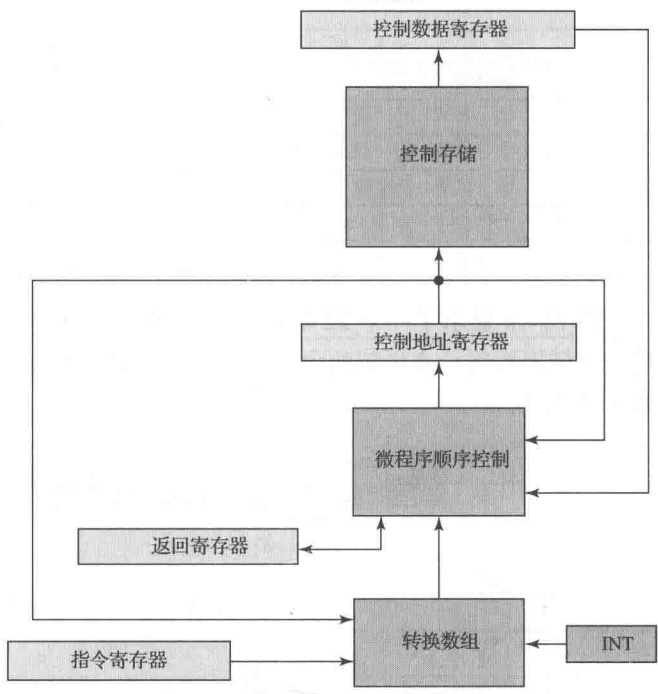


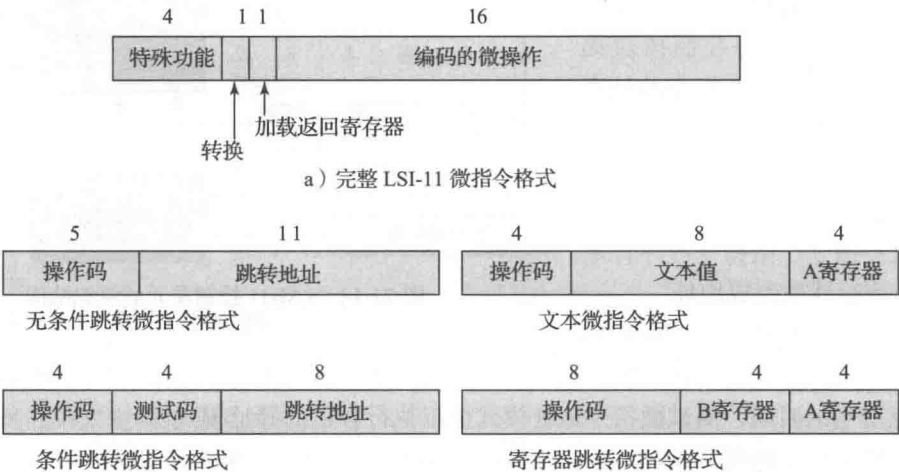
图 21-14 LSI-11 控制单元的组织结构

其所实现的 PDP-11 机器指令集非常相似。这个设计的目的是在易于编程的垂直设计约束下优化控制单元的性能。表 21-5 列出了一些 LSI-11 微指令。

表 21-5 一些 LSI-11 微指令

算术操作	通用操作
加字 (字节, 文本)	传送字 (字节)
测试字 (字节, 文本)	跳转
字 (字节) 加 1	返回
字 (字节) 加 2	条件跳转
字 (字节) 取反	设置 (重置) 标志
带条件字节递增 (递减)	加载 G 低位
带条件加字 (字节)	带条件传送字 (字节)
带进位加字 (字节)	输入 / 输出操作
带条件加数字	
减字 (字节)	输入字 (字节)
比较字 (字节, 文本)	输入状态字 (字节)
带借位减字 (字节)	读
字 (字节) 减 1	写
逻辑操作	读 (写) 字 (字节) 并加 1
	读 (写) 字 (字节) 并加 2
字 (字节, 文本) 的 “与” 操作	读 (写) 响应
测试字 (字节)	输出字 (字节, 状态)
字 (字节) 的 “或” 操作	
字 (字节) 的 “异或” 操作	
字 (字节) 位清除	
带进位字 (字节) 右 (左) 移	
字 (字节) 求补操作	

图 21-15 显示了一个 22 位的 LSI-11 微指令格式。高 4 位控制处理器板的特殊功能。转换位使转换数组检查挂起的中断。加载返回寄存器位用于微例程结束时，从返回寄存器加载微指令地址。



其余 16 位用于高度编码的微操作。其格式与机器指令非常相似，有变长操作码以及一个或多个操作数。

21.3.4 IBM 3033 微指令执行

标准 IBM 3033 控制存储器含有 4 K 个字。前半部分（0000-07FF）包含了 108 位微指令，其余部分（0800-0FFF）则用于存储 126 位微指令。格式如图 21-16 所示。

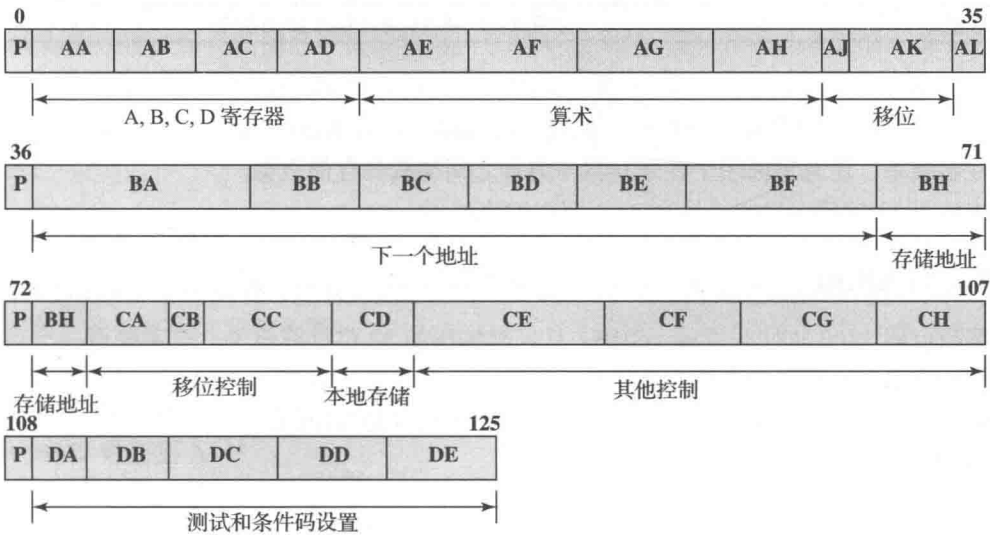


图 21-16 IBM 3033 微指令格式

尽管这是一种相当水平的格式，但还是广泛使用了编码。表 21-6 总结了这种格式的关键字段。

表 21-6 IBM 3033 微指令控制字段

ALU 控制字段		
AA(3)		从一个数据寄存器中加载 A 寄存器
AB(3)		从一个数据寄存器中加载 B 寄存器
AC(3)		从一个数据寄存器中加载 C 寄存器
AD(3)		从一个数据寄存器中加载 D 寄存器
AE(4)		把指定的 A 位路由到 ALU
AF(4)		把指定的 B 位路由到 ALU
AG(5)		在 A 输入上指定 ALU 算术运算
AH(4)		在 B 输入上指定 ALU 算术运算
AJ(1)		指定 D 或 B 输入 B 侧的 ALU
AK(4)		把算术输出路由到移位器
CA(3)		加载 F 寄存器
CB(1)		激活移位器
CC(5)		指定逻辑和进位功能
CE(7)		指定移位数
排序和分支字段		
AL(1)		结束操作并执行分支

(续)

排序和分支字段	
BA(8)	置位控制地址寄存器的高位 (00 ~ 07)
BB(4)	指定设置控制地址寄存器位 8 的条件
BC(4)	指定设置控制地址寄存器位 9 的条件
BD(4)	指定设置控制地址寄存器位 10 的条件
BE(4)	指定设置控制地址寄存器位 11 的条件
BF(7)	指定设置控制地址寄存器位 12 的条件

ALU 操作的输入来自 4 个专用的、非用户可见的寄存器：A、B、C 和 D。微指令格式包含的字段可以从用户可见寄存器加载这些寄存器、执行 ALU 功能，并指定用户可见寄存器来存储结果。还有字段用于在寄存器与内存之间加载和存储数据。

IBM 3033 的排序机制已经在 21.2 节中讨论过了。

21.4 TI 8800

德州仪器 8800 软件开发板（SDB）是可微编程的 32 位计算机卡。该系统有一个可写控制存储，在 RAM 而不是 ROM 中实现。这样的系统无法实现带有 ROM 控制存储的微编程系统的速度或密度。但是，它对开发原型以及用于教育是有用的。

8800 SDB 包含下列组件（图 21-17）：

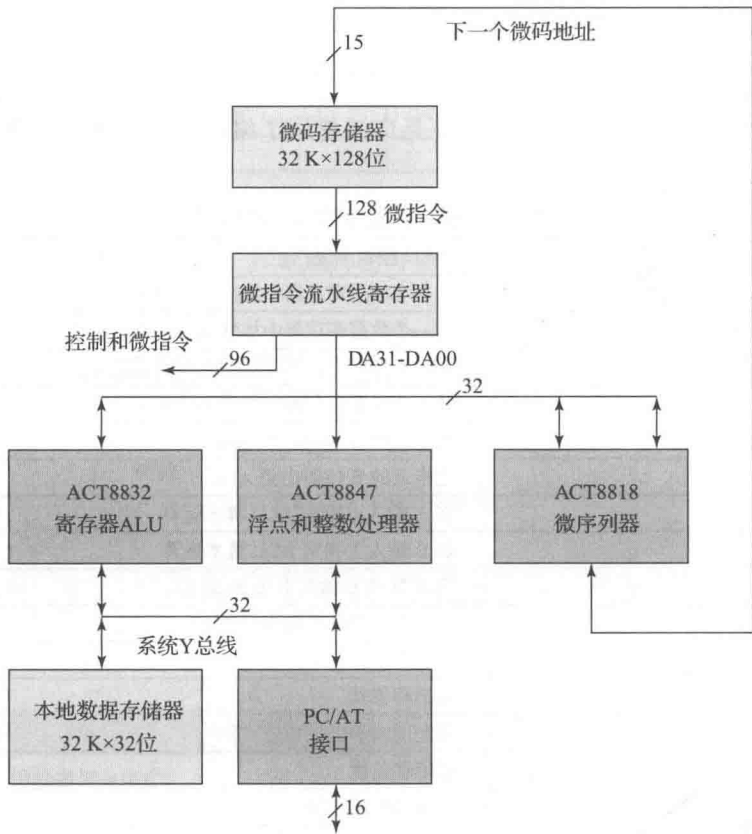


图 21-17 TI 8800 方框图

- 微码存储器
- 微序列器
- 32 位 ALU
- 浮点和整数处理器
- 本地数据存储器

两个总线连接着系统的内部组件。DA 总线提供从微指令数据字段到 ALU、浮点处理器或微序列器的数据。在后一种情况下，数据由分支指令所使用的地址组成。总线还可以用于 ALU 或微序列器，以向其他组件提供数据。系统 Y 总线把 ALU 和浮点处理器连接到本地存储器，并通过 PC 接口连接到外部模块。

这种板适合安装在与 IBM PC 兼容的主机上。主机为微码的汇编和调试提供了一个合适的平台。

21.4.1 微指令格式

8800 的微指令格式有 128 位，分成 30 个功能字段，如表 21-7 所示。每个字段包含 1 位或多位，这些字段又分为 5 类：

- 板控
- 8847 浮点和整数处理器芯片
- 8832 寄存器 ALU
- 8818 微序列器
- WCS 数据字段

表 21-7 TI 8800 微指令格式

字段编号	位数	描述
板控		
1	5	选择条件码输入
2	1	允许 / 禁止外部 I/O 请求信号
3	2	允许 / 禁止本地数据存储器读 / 写操作
4	1	加载状态 / 不加载状态
5	2	确定单元驱动 Y 总线
6	2	确定单元驱动 DA 总线
8847 浮点和整数处理芯片		
7	1	C 寄存器控制：时钟，无时钟
8	1	为 Y 总线选择最低或最高有效位
9	1	C 寄存器数据源：ALU，多路复用器
10	4	为 ALU 和 MUL 选择 IEEE 或 FAST 模式
11	8	为数据操作数选择源：RA 寄存器、RB 寄存器、P 寄存器、5 寄存器、C 寄存器
12	1	RB 寄存器控制：时钟，无时钟
13	1	RA 寄存器控制：时钟，无时钟
14	2	数据源的确认
15	2	允许 / 禁止流水线寄存器
16	11	8847 ALU 功能
8832 寄存器 ALU		
17	2	写允许 / 禁止数据输出到选择的寄存器：高半部分，低半部分

(续)

字段编号	位数	描述
8832 寄存器 ALU		
18	2	选择寄存器文件数据源: DA 总线、DB 总线、ALU Y MUX 输出、系统 Y 总线
19	3	移位指令修改器
20	1	进位: 强制, 非强制
21	2	设置 ALU 配置模式: 32、16 或 8 位
22	2	选择 5 多路复用器的输入: 寄存器文件、DB 总线、MQ 寄存器
23	1	选择 R 多路复用器的输入: 寄存器文件、DA 总线
24	6	选择文件 C 中的寄存器进行写操作
25	6	选择文件 B 中的寄存器进行读操作
26	6	选择文件 A 中的寄存器进行写操作
27	8	ALU 功能
8818 微序列器		
28	12	控制到 8818 的输入信号
WCS 数据字段		
29	16	可写控制存储数据字段的最高有效位
30	16	可写控制存储数据字段的最低有效位

如图 21-17 所示, WCS 数据字段的 32 位送入 DA 总线, 作为提供给 ALU、浮点处理器或微序列器的数据。微指令的其他 96 位(字段 1~27)是控制信号, 直接提供给适当的模块。为了简单起见, 图 21-17 中没有显示其他这些连接。

前 6 个字段处理与板控有关的操作, 而不是控制单个组件。控制操作包括以下内容:

- 为排序器控制选择条件码。字段 1 的第一位表示把条件标志设为 1 或 0, 其余 4 位指示要设置的标志。
- 向 PC/AT 发送 I/O 请求。
- 允许本地数据存储器读 / 写操作。
- 确定驱动系统 Y 总线的单元。选择连接到该总线的 4 个设备(图 21-17)中的一个。

最后 32 位是数据字段, 它包含了针对特定微指令的信息。

微指令的其余字段最好是在它们所控设备的上下文中讨论。在本节的其余部分, 我们将讨论微序列器和寄存器 ALU。浮点单元没有引入任何新概念, 因此不再介绍。

21.4.2 微序列器

8818 微序列器的主要功能是为微程序生成下一条微指令的地址。这个 15 位地址被提供给微码存储器(图 21-17)。

下一个地址选择自 5 个来源:

- 1) 微程序计数器(MPC)寄存器, 用于重复(重用相同的地址)和继续(地址加 1)指令。
- 2) 栈, 它支持微程序子例程调用以及迭代循环和中断返回。
- 3) DRA 和 DRB 端口提供来自外部硬件的两条额外路径, 通过它们可以生成微程序地址。这两个端口分别连接到 DA 总线的高 16 位和低 16 位。这使得微序列器能从当前微指令的 WCS 数据字段, 或者从 ALU 的计算结果中获得下一条指令的地址。
- 4) 寄存器计数器 RCA 和 RCB, 可用于其他地址存储。

5) 到双向 Y 端口的外部输入, 以支持外部中断。

图 21-18 是 8818 的逻辑方框图。该设备包含以下主要功能组:

- 16 位微程序计数器 (MPC), 由寄存器和增量器组成。
- 两个寄存器计数器 RCA 和 RCB, 用于对循环和迭代进行计数, 存储分支地址, 或驱动外部设备。
- 65 字 $\times$ 16 位的栈, 允许微程序子例程调用和中断。
- 中断返回寄存器和 Y 输出支持微指令级的中断处理。
- Y 输出多路复用器, 通过它从 MPC、RCA、RCB, 外部总线 DRA 和 RDB 或者栈选择下一个地址。

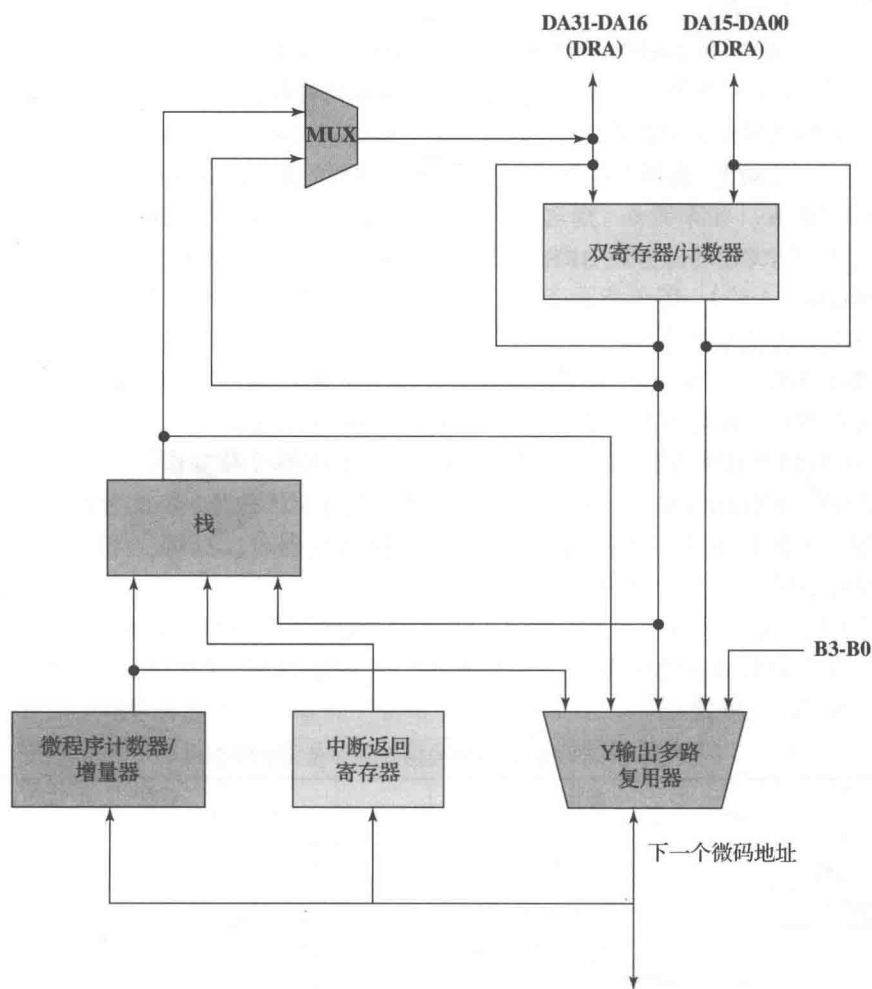


图 21-18 TI 8818 微序列器

寄存器 / 计数器 寄存器 RCA 和 RCB 可以从 DA 总线加载, 也可以从当前微指令或者 ALU 输出加载。这些值可以用作计数器来控制执行流, 并在访问时自动递减。这些值还能用作微指令地址, 以提供给 Y 输出多路复用器。支持单个微指令周期内对两个寄存器的独立控制, 但两个寄存器同时递减的情况除外。

栈 栈允许多级嵌套调用或中断, 并可以用于支持分支和循环。请记住, 这些操作是指

控制单元，而不是整个处理器，所涉及地址是控制存储器中微指令的地址。

可以进行 6 种栈操作：

- 1) 清除，将栈指针设为 0，清空栈。
- 2) 出栈，栈指针递减。
- 3) 入栈，把 MPC、中断返回寄存器或 DRA 总线的内容放入栈，栈指针递增。
- 4) 读，使读指针指示的地址在 Y 输出多路复用器上可用。
- 5) 保持，使栈指针地址保持不变。
- 6) 加载栈指针，把 DRA 的低 7 位输入栈指针。

微序列器的控制 微序列器主要由当前微指令的 12 位字段控制，即字段 28（表 21-7）。

该字段包含如下子字段：

- OSEL（1 位） 输出选择。决定哪个值将被放到多路复用器的输出，以送入 DRA 总线（图 21-18 左上角）。选择输出来自栈或者来自寄存器 RCA。然后，DRA 将作为 Y 输出多路复用器或寄存器 RCA 的输入。
- SELDR（1 位） 选择 DR 总线。如果置 1，该位选择外部 DA 总线作为 DRA/DRB 总线的输入。如果置 0，则选择 DRA 多路复用器的输出到 DRA 总线（由 OSEL 控制），以及 RCB 的内容到 DRB 总线。
- ZEROIN（1 位） 用于指示条件分支。然后，微序列器的行为将取决于字段 1（表 21-7）中选择的条件码。
- RC2 ~ RC0（3 位） 寄存器控制。这些位决定寄存器 RCA 和 RCB 内容的变化。每个寄存器可以保持不变、递减或者从 DRA/DRB 总线加载。
- S2 ~ S0（3 位） 栈控制。这些位决定要执行的是哪个栈操作。
- MUX2 ~ MUX0 输出控制。这些位与条件码（如果使用）一起控制 Y 输出多路复用器，由此控制下一条微指令的地址。多路复用器可以从栈、DRA、DRB 或 MPC 中选择其输出。

这些位可以由程序员单独设置。但是，通常不会这样做。相反，程序员使用助记符，这些助记符等同于通常所需的位模式。表 21-8 列出了字段 28 的 15 个助记符。微码汇编器把它们转换成适当的位模式。

表 21-8 TI 8818 微序列器微指令位（字段 28）

助记符	数值	描述
RST8818	000000000110	重启指令
BRA88181	011000111000	分支到 DRA 指令
BRA88180	010000111110	分支到 DRA 指令
INC88181	000000111110	连续指令
INC88180	001000001000	连续指令
CAL88181	010000110000	跳转到由 DRA 指定地址的子例程
CAL88180	010000101110	跳转到由 DRA 指定地址的子例程
RET8818	000000011010	从子例程返回
PUSH8818	000000110111	把中断返回地址入栈
POP8818	100000010000	从中断返回
LOADDRA	000010111110	从 DA 总线加载 DRA 计数器
LOADDRB	000110111110	从 DA 总线加载 DRB 计数器

(续)

助记符	数值	描述
LOADDRAB	000110111100	加载 DRA/DRB
DECRDRA	010001111100	如果非零, 则递减 DRA 计数器并分支
DECRDRB	010101111100	如果非零, 则递减 DRB 计数器并分支

举个例子, 如果当前被选择的条件码为 1, 那么使用指令 INC88181 来使得顺序的下一条指令被选中。从表 21-8 中, 我们可以得到:

INC88181 = 000000111110

它会直接被译码为

- OSEL=0: 选择 RCA 作为 DRA 输出到 MUX 的输出; 在这种情况下, 所选内容不相关。
- SELDR=0: 如前定义; 同样, 这与该指令无关。
- ZEROIN=0: 与 MUX 的值相结合, 表示不应分支。
- R=000: 保留 RA 和 RC 的当前值。
- S=111: 保留栈的当前状态。
- MUX=110: 当条件码 =1 时, 选择 MPC; 当条件码 =0 时, 选择 DRA。

21.4.3 寄存器 ALU

8832 是一个 32 位 ALU, 带有 64 个寄存器, 可以配置为 4 个 8 位 ALU、2 个 16 位 ALU 和 1 个 32 位 ALU。

8832 由组成微指令字段 17 到字段 27 (表 21-7) 的 39 位来控制; 它们作为控制信号提供给 ALU。此外, 如图 21-17 所示, 8832 到 32 位 DA 总线和 32 位系统 Y 总线有外部连接。来自 DA 的输入可以同时作为输入数据提供给 64 字寄存器文件和 ALU 逻辑模块。来自系统 Y 总线的输入提供给 ALU 逻辑模块。ALU 和移位操作的结果是给 DA 总线或系统 Y 总线的输出。结果也可以反馈到内部寄存器文件。

3 个 6 位地址端口允许同时在寄存器文件中执行取双操作数和写单操作数。MQ 移位器和 MQ 寄存器也可以配置为独立运行, 以实现双精度 8 位、16 位和 32 位移位操作。

每个微指令的字段 17 到字段 26 控制数据在 8832 内部以及在 8832 与外部环境之间的流动方式。字段如下:

17. **写使能**。这两位指示写 32 位、16 位最高有效位、16 位最低有效位, 或不写入寄存器文件。目标寄存器由字段 24 定义。

18. **选择寄存器文件数据源**。如果在寄存器文件中发生了写操作, 这两位指定源: DA 总线、DB 总线、ALU 输出或系统 Y 总线。

19. **移位指令修改符**。指定在移位指令期间, 提供结束填充位和读取移位位的选项。

20. **进位**。该位指示是否为了此操作把位进到 ALU 中。

21. **ALU 配置模式**。8832 可以配置为 1 个 32 位 ALU、2 个 16 位 ALU 或 4 个 8 位 ALU。

22. **S 输入**。ALU 逻辑模块输入由两个内部多路复用器提供, 称为 S 和 R 多路复用器。本字段选择由 S 多路复用器提供的输入: 寄存器文件、DB 总线或 MQ 寄存器。源寄存器由字段 25 定义。

23. **R 输入**。选择由 R 多路复用器提供的输入: 寄存器文件或 DA 总线。

24. **目标寄存器**。寄存器文件中的寄存器地址，用于目的操作数。
25. **源寄存器**。寄存器文件中的寄存器地址，用于由 S 多路复用器提供的源操作数。
26. **源寄存器**。寄存器文件中的寄存器地址，用于由 R 多路复用器提供的源操作数。

最后，字段 27 是一个 8 位操作码，指定 ALU 要执行的算术或逻辑功能。表 21-9 列出了可以执行的不同操作。

表 21-9 TI 8832 寄存器 ALU 指令字段（字段 27）

组 1		功能
ADD	H#01	$R + S + Cn$
SUBR	H#02	$(NOT\ R) + S + Cn$
SUBS	H#03	$R = (NOT\ S) + Cn$
INSC	H#04	$S + Cn$
INCNS	H#05	$(NOT\ S) + Cn$
INCR	H#06	$R + Cn$
INCNR	H#07	$(NOT\ R) + Cn$
XOR	H#09	$R\ XOR\ S$
AND	H#0A	$R\ AND\ S$
OR	H#0B	$R\ OR\ S$
NAND	H#0C	$R\ NAND\ S$
NOR	H#0D	$R\ NOR\ S$
ANDNR	H#0E	$(NOT\ R)\ AND\ S$
组 2		功能
SRA	H#00	单精度算术右移
SRAD	H#10	双精度算术右移
SRL	H#20	单精度逻辑右移
SRLD	H#30	双精度逻辑右移
SLA	H#40	单精度算术左移
SLAD	H#50	双精度算术左移
SLC	H#60	单精度循环左移
SLCD	H#70	双精度循环左移
SRC	H#80	单精度循环右移
SRCD	H#90	双精度循环右移
MQSRA	H#A0	MQ 寄存器算术右移
MQSRL	H#B0	MQ 寄存器逻辑右移
MQSLL	H#C0	MQ 寄存器逻辑左移
MQSLC	H#D0	MQ 寄存器循环左移
LOADMQ	H#E0	加载 MQ 寄存器
PASS	H#F0	把 ALU 传递给 Y（无移位操作）
组 3		功能
SET1	H#08	置位 1
Set0	H#18	置位 0
TB1	H#28	测试位 1
TB0	H#38	测试位 0
ABS	H#48	绝对值
SMTc	H#58	原码 / 二进制补码

(续)

组 3		功能
ADDI	H#68	加立即数
SUBI	H#78	减立即数
BADD	H#88	把 R 字节加到 S
BSUBS	H#98	从 R 中字节减去 S
BSUBR	H#A8	从 S 中字节减去 R
BINCS	H#B8	字节递增 S
BINCNS	H#C8	字节递增 S 反
BXOR	H#D8	R 和 S 的字节 XOR
BAND	H#E8	R 和 S 的字节 AND
BOR	H#F8	R 和 S 的字节 OR
组 4		功能
CRC	H#00	循环冗余字符累加器
SEL	H#10	选择 S 或 R
SNORM	H#20	单长度规格化
DNORM	H#30	双长度规格化
DIVRF	H#40	除法余数修正
SDIVQF	H#50	有符号除法商修正
SMULI	H#60	有符号乘法迭代
SMULT	H#70	有符号乘法终止
SDIVIN	H#80	有符号除法初始化
SDIVIS	H#90	有符号除法开始
SDIVI	H#A0	有符号除法迭代
UDIVIS	H#B0	无符号除法开始
UDIVI	H#C0	无符号除法迭代
UMULI	H#D0	无符号乘法迭代
SDIVIT	H#E0	有符号除法终止
UDIVIT	H#F0	无符号除法终止
组 5		功能
LOADFF	H#0F	加载除法 /BCD 触发器
CLR	H#1F	清除
DUMPFF	H#5F	输出除法 /BCD 触发器
BCDBIN	H#7F	BCD 到二进制
EX3BC	H#8F	余 3 字节校正
EX3C	H#9F	余 3 字校正
SDIVO	H#AF	有符号除法溢出测试
BINEX3	H#DF	二进制到余 3 码
NOP32	H#FF	无操作

作为用于指定字段 17 到字段 27 的编码示例, 考虑这样一条指令: 把寄存器 1 的内容加到寄存器 2, 并把结果放入寄存器 3。符号指令为

CONT11 [17], WELH, SELRYFYMx, [24], R3, R2, R1, PASS + ADD

汇编器将把其转换为合适的位模式。指令的各个组成部分可以描述如下:

- CONT11 是基本 NOP 指令。
- 字段 [17] 更改为 WELH (写使能、低和高), 以便将 32 位寄存器写入。
- 字段 [18] 更改为 SELRFYMX, 以选择来自 ALU、Y、MUX 输出的反馈。
- 字段 [24] 更改为指定寄存器 R3 作为目标寄存器。
- 字段 [25] 更改为指定寄存器 R2 作为一个源寄存器。
- 字段 [26] 更改为指定寄存器 R1 作为一个源寄存器。
- 字段 [27] 更改为指定的 ADD 的 ALU 操作。ALU 移位指令 PASS; 因此, ALU 输出不会被移位器移位。

关于符号表示法有几点要说明。没有必要为连续字段指定字段编号。也就是说

```
CONT11 [17],WELH,[18],SELRFYMX
```

可以写为

```
CONT11 [17],WELH,SELRFYMX
```

因为 SELRFYMX 在字段 18 中。

表 21-9 组 1 中的 ALU 指令必须总是与组 2 一起使用。组 3 到组 5 中的 ALU 指令不能与组 2 一起使用。

21.5 关键词、复习题和练习题

关键词

control memory: 控制存储器

control word: 控制字

firmware: 固件

hard microprogramming: 硬微编程

horizontal microinstruction: 水平微指令

microinstruction encoding: 微指令编码

microinstruction execution: 微指令执行

microinstruction sequencing: 微指令排序

microinstructions: 微指令

microprogram: 微程序

microprogrammed control unit: 微程序控制单元

microprogramming language: 微编程语言

soft microprogramming: 软微编程

unpacked microinstruction: 非压缩微指令

vertical microinstruction: 垂直微指令

复习题

21.1 控制单元的硬连线实现与微程序实现之间有何差异?

21.2 如何解释水平微指令?

21.3 控制存储器的目的是什么?

21.4 水平微指令执行的典型顺序是什么?

21.5 水平微指令与垂直微指令之间有何差异?

21.6 微程序控制单元执行的基本任务是什么?

21.7 压缩和非压缩微指令之间有何差异?

21.8 硬微编程与软微编程之间有何差异?

21.9 功能型与资源型编码之间有何差异?

21.10 请列出微程序设计的一些常见应用。

练习题

21.1 描述 Wilkes 设计的假设机器中乘法指令的实现。使用语言叙述和流程图。

21.2 假设微指令集中一条微指令有如下符号形式：

$$\text{IF } (AC_0 = 1) \text{ THEN } CAR \leftarrow (C_{0-6}) \text{ ELSE } CAR \leftarrow (CAR) + 1$$

其中， AC_0 是累加器的符号位， C_{0-6} 是该微指令的前 7 位。使用该微指令，编写微程序实现“分支寄存器负号”（BRM）指令，当 AC 为负时分支。假设微指令的位 C_1 到位 C_n 指定一组并行的微操作。用符号表示程序。

21.3 简单处理器的指令周期有 4 个主要阶段：取指、间接、执行和中断。在硬连线实现中，两个 1 位标志指定当前阶段。

a. 为什么需要这些标志？

b. 为什么在微程序控制单元中不需要这些位？

21.4 考虑图 21-7 中的控制单元。假设控制存储器为 24 位宽。微指令格式的控制部分分为两个字段。13 位的微操作字段指定要执行的微操作。地址选择字段根据标志指定一个条件，该条件将导致分支。有 8 个标志。

a. 地址选择字段需要多少位？

b. 地址字段需要多少位？

c. 控制存储器大小是多少？

21.5 在前一题的情况下，如何实现无条件分支？如何避免分支，也就是说，描述一条不指定任何分支的微指令，不论是有条件的还是无条件的。

21.6 我们希望为每个机器指令例程提供 8 个控制字。机器指令操作码为 5 位，控制存储器有 1024 个字。建议把指令寄存器映射到控制地址寄存器。

21.7 使用一种编码微指令格式。展示如何把 9 位微操作字段划分成子字段，以指定 46 种不同的动作。

21.8 处理器包含 16 个寄存器、一个有 16 种逻辑和 16 种算术功能的 ALU、一个有 8 种操作的移位器，内部处理器总线连接所有这些组件。设计一种微指令格式，为该处理器指定各种微操作。